PyPTO算子编程专场

CANN开发者Meetup

2026.10.24　中国·上海

主办单位： CANN开源社区　**协办单位：** AtomGit



CANN开发者Meetup – PyPTO专场议题

1. PyPTO编程范式和框架能力介绍	14：00 – 14：20
2. 基于 CANNBot的PyPTO算子自动生成及性能优化	14：20 – 14：40
3. PyPTO实践：基于Tensor模式的Ling-3.0-flash KDA算子开发	14：40 – 15：00
4. PyPTO实践：基于Pro模式的FlexAttention算子开发	15：00 – 15：20
5. PyPTO端侧能力介绍及实践	15：20 – 15：40
6. 茶歇	15：40 – 16：00
7. PTO ISA：面向昇腾的硬件抽象和跨代兼容	16：00 – 16：20
8. PTO 编译：面向Tile抽象的编译优化技术	16：20 – 16：40
9. “开发者面对面”互动交流	16：40 – 17：00

CANN

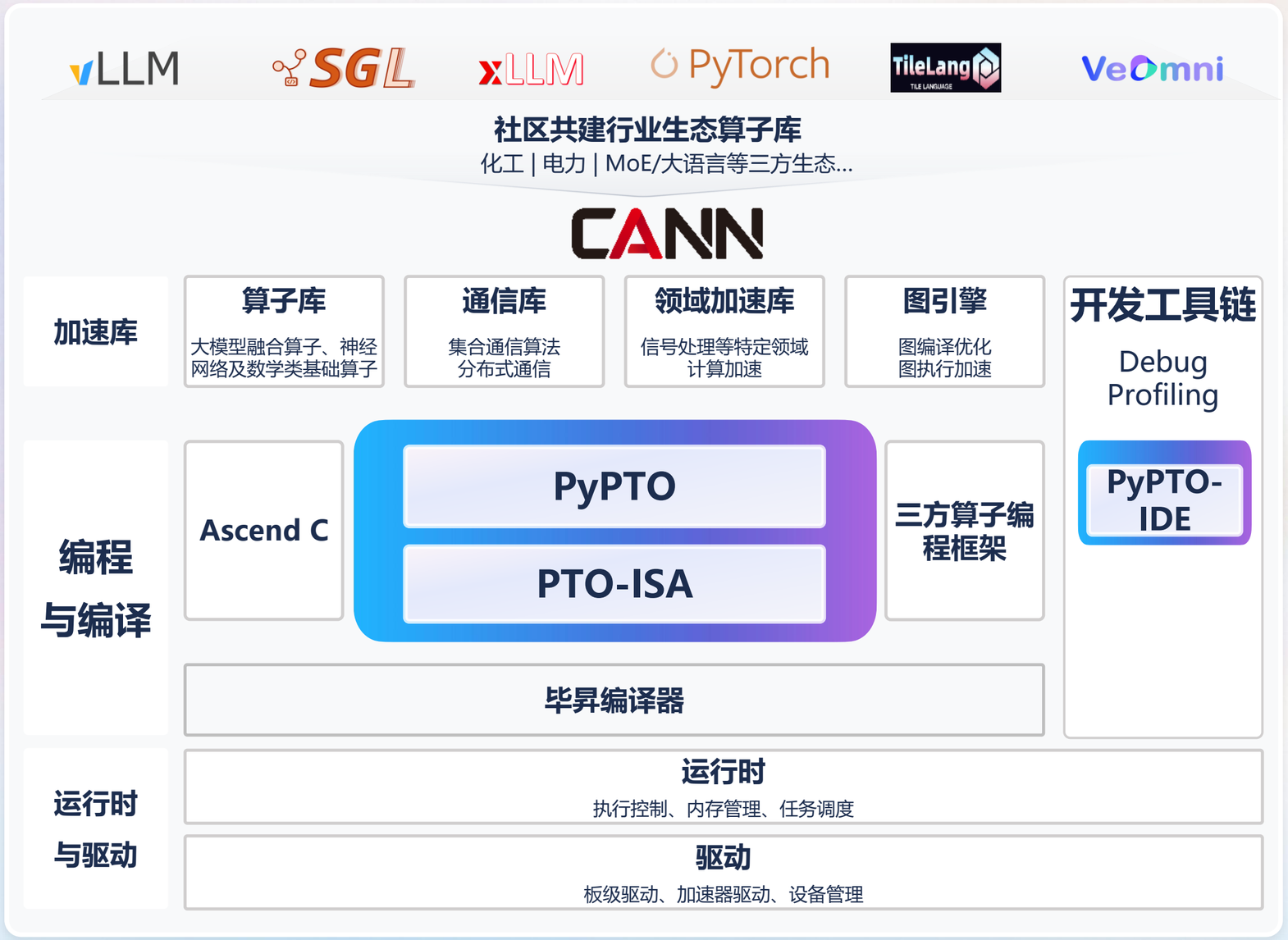
CANN

PyPTO编程范式和框架能力介绍

CyberTree



PyPTO是昇腾原生的高效算子编程框架



什么是PyPTO

昇腾原生高效算子编程
贯穿硬件,指令集,框架设计

定位复杂融合算子开发
简化复杂融合算子开发流程

兼顾易用性和性能目标
优化开发效率和执行效率

主要设计思想

编程范式
Tensor和Tile为核心的PTO编程范式

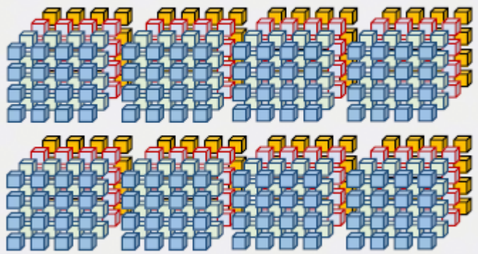
虚拟指令集
屏蔽硬件差异，支持跨代跨平台演进

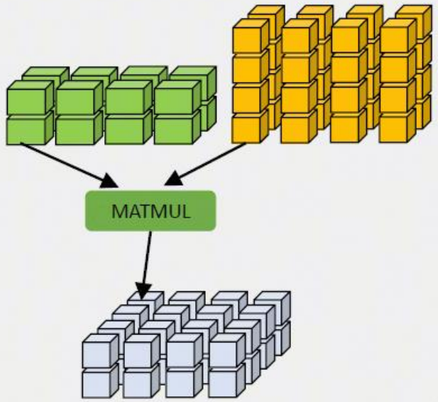
Python原生
提供Python原生接口降低开发门槛

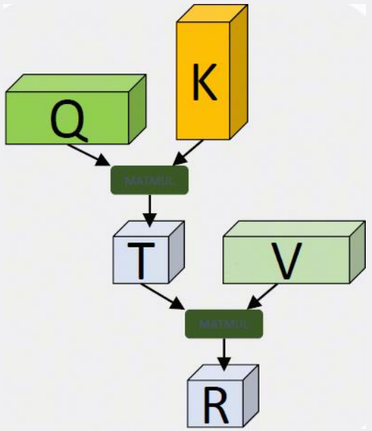
编程易用性
全流程白盒调试调优和Agent自动算子生成

CANN

PyPTO 采用围绕Tensor和Tile的编程范式







Scalar Operation (32bit)

- ✔ 表达灵活，适用范围广
- ✘ 控制与硬件实现复杂
- ✘ 计算冗余较多，执行效率偏低

Tile Operation (16 KB)

- ✔ 适配硬件存储层级
- ✔ 匹配核内 Buffer 容量
- ✔ 抽象简洁，控制复杂度适中

Tensor Operation (256 MB)

- ✔ 表达直观，适配 Python 编程
- ✘ 数据规模较大，难以片上存储
- ⊖ 经编译映射为 Tile 执行

CANN

复杂融合算子开发的核心目标和挑战

目标

提高**访存**效率

提高执行**并发**

降低**调度**开销

挑战

存储
结构复杂，容量有限

计算
异构执行单元难以协调

调试
正确性和调优困难

方法

围绕**Tensor/Tile**的计算表达，编译和执行

白盒化调试调优**工具链**
Agent自动算子生成

虚拟指令集
跨代跨平台兼容

复杂融合算子通过融合执行，减少中间数据搬运与 Kernel Launch开销，提升并行度，缓解内存墙。

CANN

Tensor和Tile两种编程模式 兼顾易用性与优化性能

Tensor 模式

易用性 – 提高开发效率

- **编程界面** 抽象程度更高，开发者写计算逻辑
- **编译器** 计算和数据逐步映射到内存和计算
- **运行时** 动态调度，提升执行单元利用率

```
@pypto.frontend.jit
def add_kernel(
    a: pypto.Tensor[[64], pypto.DT_FP16],
    b: pypto.Tensor[[64], pypto.DT_FP16],
    out: pypto.Tensor[[64], pypto.DT_FP16],
):
    out[:] = a + b
```

Tile (Pro) 模式

优化性能 – 提高执行效率

- **编程界面** 暴露底层细节计算和内存管理控制权
- **编译器** 计算和内存管理表达翻译成可执行代码
- **运行时** 高效调度，高效执行

```
@pl.jit(auto_mutex=True)
def add_kernel(
    x: pl.Tensor[[64], pl.DT_FP16],
    y: pl.Tensor[[64], pl.DT_FP16],
    out: pl.Tensor[[64], pl.DT_FP16],
):
    tt = pl.TileType(
        shape=[64], dtype=pl.DT_FP16, target_memory=pl.MemorySpace.Vec
    )
    t_x = pl.make_tile_group(type=tt, addrs=0x0000, mutex_ids=[0])
    t_y = pl.make_tile_group(type=tt, addrs=0x2000, mutex_ids=[1])
    t_out = pl.make_tile_group(type=tt, addrs=0x4000, mutex_ids=[2])

    with pl.section_vector():
        pl.load(t_x.current(), x, [0])
        pl.load(t_y.current(), y, [0])
        pl.add(t_out.current(), t_x.current(), t_y.current())
        pl.store(t_out.current(), out, [0])
```

CANN

Tensor和Tile编程模式对比

Tensor 模式

易用性 – 提高开发效率

- **基于Tensor**的前端DSL设计
- 开发者**不感知硬件架构**
- 编译器**自动内存管理**
- 编译器**自动计算和数据切分**
- 编译器自动核间同步
- PTO ISA适配后端平台
- 适配**训练场景**优化开发效率

Tile (Pro) 模式

优化性能 – 提高执行效率

- **基于Tile**的前端DSL设计
- 开发者**感知硬件架构**
- 开发者**手动内存管理**
- 开发者**手动计算和数据切分**
- 编译器自动核间同步
- PTO ISA适配后端平台
- 适配**推理场景**优化执行效率

CANN

PyPTO算子开发工作流

算子定义

梳理计算流
明确算子原型

算子实现

计算模块实现
完成计算流组装

编译和执行

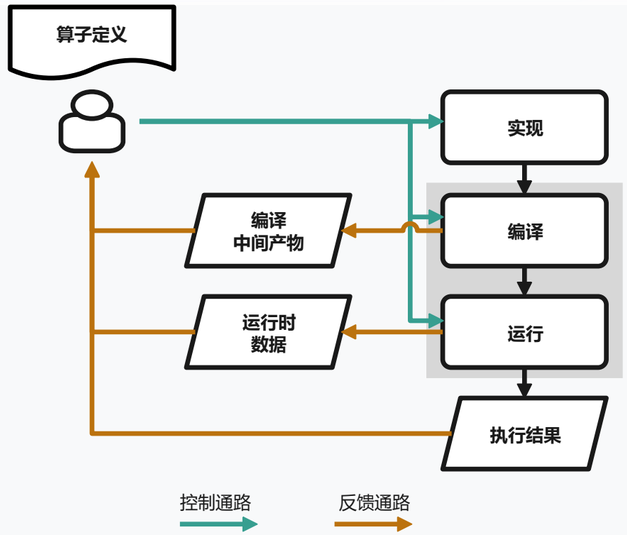
PyPTO框架编译
仿真或上板执行

检查编译和执行结果

获取中间和执行数据
调整编译执行选项

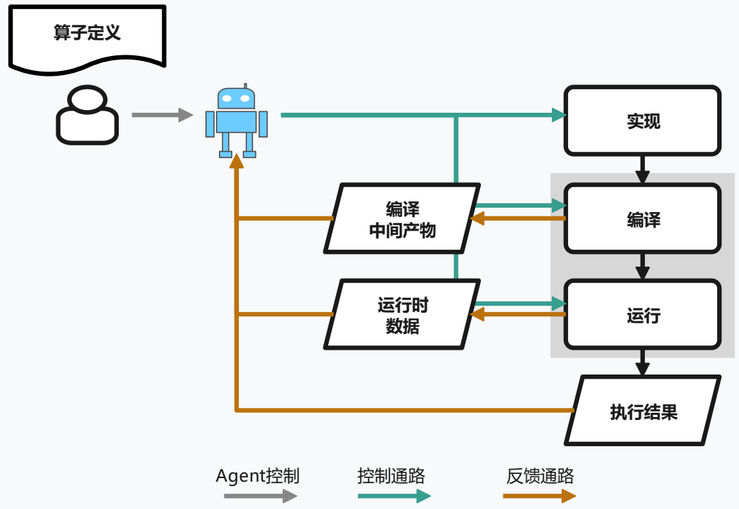
Man In The Loop

开发者完全掌控 PyPTO 算子开发全流程
进行迭代优化实现



Agent In The Loop

开发者通过 Prompt 输入算子定义
借助 PyPTO Agent 驱动算子开发流程



CANN

PyPTO开源社区

开源时间节点



开源代码仓

PyPTO框架	atomgit.com/pypto
样例和Agent框架	atomgit.com/pypto-gym
PyPTO IDE	atomgit.com/pypto-vscode
PTO指令集	atomgit.com/pto-isa

PyPTO文档

pypto.atomgit.com



CANN

THANKS



CANN

基于 CANNBot 的 PyPTO 算子自动生成及性能优化

诸葛寻优



复杂大模型算子的开发难点

一个 shape 跑通之后，还要守住其他规格的精度、同步和真实调用表现

01 数学语义与接口

group_list：按 M 还是 K 分组?

Y_g = Yref_g + s1_g·s2_g·P_g

分组轴 / 广播 / Cache 写入语义

02 形状与内存布局

“规则形状都对，N=80 就错?”

N_pad = 32 * ceil(N / 32)

padding / 尾块 / stride / 片上资源

03 低精度数值

“scale 对了，y 为何还不一致?”

|y-ref| ≤ atol + rtol·|ref|

舍入 / FP8 打包 / 逐输出验收

复杂算子开发

改一处，重验多处

算对 · 跑稳 · 够快

04 并行与同步

“编译过了，设备却不返回?”

matmul(..., phase=Final)

多核分工 / 数据依赖 / 累加收尾

05 性能与瓶颈

“加核、加批量，为什么更慢?”

tile = worker + i * workers

避免重复计算 / 搬运瓶颈转移

06 验证与业务接入

“内核快了，接进模型反而慢?”

QKV t183: kernel 1.46× / e2e 0.38×

独立 Golden / 多规格回归 / 计时协议

“这个 shape 过了，换一个 shape 怎么又错了?”

案例来自 pure_mode_1030。代码与公式为简化示意；QKV 两项加速比采用不同计时协议，详见讲稿。

CANNBot PyPTO 算子自动生成及性能优化

统一四阶段任务视图，每阶段检查与验证，成果与证据一起落盘

共同 Prompt

实现并优化 softmax：float16，[4, 2048]，沿最后一维计算。
目标 NPU 独立验证并调优，保存源码、测试、报告、耗时与 Token。

PyPTO Tensor Agent

pypto-op-orchestrator

PyPTO Pro Agent

pypto-pro-op-orchestrator

01
理解需求
公式、Shape / Dtype
精度与性能验收条件

检查：规格、API 可行性
接口与输入约定

→

02
准备参考与环境
独立 Golden、测试合同
边界用例与设备环境

验证：Golden 自检
测试用例与设备可用

→

03
代码编写与测试
实现、编译、运行
失败定位与修复重验

验证：独立精度对照
最终结构与布局检查

→

04
性能优化
瓶颈分析、候选实验
基于实测判断收益

验证：固定计时口径
独立回归、采纳或回退

每阶段检查留证，失败反馈修复；改动后重新验证，最终按冻结合同验收

成果落盘 custom/softmax/

源码 / 测试 / 报告 / 状态 / 成本

CANNBot PyPTO Agent 仓库路径

准备匹配的 NPU 环境，选择开发路线，再提交第一句需求

01 准备环境与仓库

git clone https://gitcode.com/cann/pypto-gym.git

昇腾 NPU · 匹配 CANN 与 PyPTO / PyPTO-Pro · OpenCode

02 PyPTO Tensor Agent

使用 PyPTO API 开发算子

pypto-op-orchestrator

bash init.sh project opencode
opencode

02 PyPTO Pro Agent

默认 Scriptor：计算与数据流描述

pypto-pro-op-orchestrator

bash init.sh project opencode
opencode

03 输入第一句需求

“开发 softmax，输入 float16，shape [4, 2048]，生成实现并验证精度。”

写清语义、输入约束、精度与性能目标；入口文档：各目录 quickstart.md

入口与命令依据原图；详细安装条件、完整命令及示例见本页讲稿。

CANNBot PyPTO-Pro 算子开发实践

grouped_matmul_activation_quant：原始 Prompt、开发过程与交付结果

01 准备源码与运行环境

在代码 Agent 中打开算子仓库
准备可运行环境与官方 golden / cases

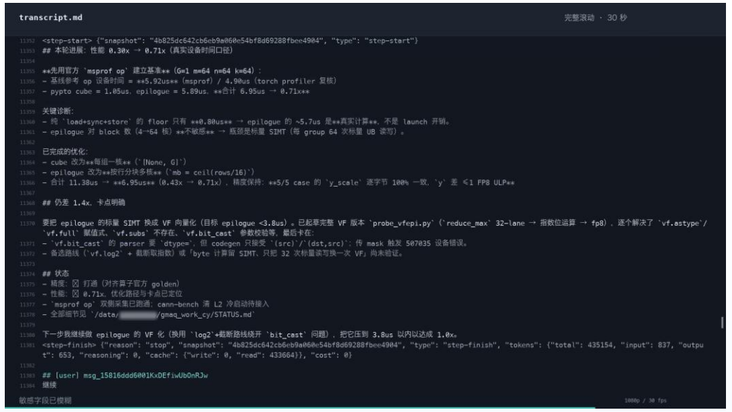
02 输入任务 Prompt

请充分理解 /data/[redacted]/ops-transformer 下的 attention/grouped_matmul_activation_quant 这个算子，理解 这个算子在 基线参考 框架下是如何实现的，结构是什么样子的，现有的测试cases都有哪些。然后看下这个算子的作用/结构/公式，要客观分析，充分理解并验证，不要出错，介绍时应该精炼简明准确易懂，生成一个算子文档。

之后，在当前环境下，即conda [redacted] 下，配置 基线参考 运行环境并且跑通这个 [redacted] 基线参考 算子。

之后，按照 cannbot-skills/ops/pypto-pro-kernel 这个 skill 的流程，按照这个 基线参考 算子的功能与 NPU 流水，将其翻译转化为 对应的 pypto-pro kernel。目标是在精度打通的基础上使得各个 基线参考 默认运行的 cases 都达到但不限于 1.0 倍的 基线参考 的 performance，两边的性能采集方式严格按照 基线参考 官方给出的 msprof 采集方式来，额外注意要默认增加 https://gitcode.com/cann/cann-bench 中 对 pypto-pro kernel 采集性能前的 清 L2 冷启动机制。 注意：当前环境下可能有多个agent 在做相同的任务，你不要参考他们的产物，互相之间不要影响，就做自己的，可以搞一个单独的工作目录。

03 查看开发过程



点击画面播放 30 秒全过程

生成 kernel、测试脚本与算子文档

```
grouped_matmul_activation_quant/
├── SUMMARY.md                    ← 归纳汇总（md，含全部 4 项）
├── code/                         ← pypto-pro kernel + 配套脚本（8 个文
│   ├── gmaq_kernel.py            ★ kernel（cube + epilogue，按 case 特
│   ├── test.py / ref_ascendc.py 精度对照（官方 golden）
│   ├── final_measure.py / devtime.py / analyze.py  设备时间与加速比
│   └── load_op.py / env.sh
├── generated_kernels/            ← 37 个变体的即时生成 kernel 源码（.cpp
└── operator_doc/GroupedMatmulActivationQuant.md   ← 算子文档
```

04 核验精度与性能结果

5/5 用例精度通过

y_scale 字节一致
y 偏差 ≤1 FP8 ULP

1.82–2.38×

torch_npu profiler 设备时间加速比

case	基线参考 (μs)	pypto cube+epi (μs)	加速比
1,64,64,64	5.16	0.99+1.31=2.29	2.25×
4,32,128,128	5.08	1.19+1.60=2.79	1.82×
4,64,64,256	5.59	1.32+1.58=2.90	1.93×
4,128,256,256	12.36	2.95+3.23=6.18	2.00×
2,64,128,64	6.60	1.17+1.60=2.77	2.38×

来源：transcript.md 原始会话。结果覆盖 MXFP8/E4M3、scale_alg=0；完整可复制 Prompt 见备注。

开发者实践

PyPTO Pro Agent

原始会话实录（已脱敏）

transcript.md　　同步回放 · 60 秒

```
# Session transcript — ses_ea900b430ffe0PYsJyTdUoa4nl
- directory: `/data/[redacted]/pypto-gym-cy/cannbot-skills/plugins-official/pypto-pro-op-orchestrator`
- title: grouped_matmul_activation_quant 转 pypto-pro kernel
- agent: build  model: {"id":"deepseek-v4.1-flash","providerID":"my-cannbot"}
- messages: 257  parts: 1074


## [user] msg_156ff4c4e001YNrU0Si09fJlbf
请充分理解 /data/[redacted]/ops-transformer 下的 attention/grouped_matmul_activation_quant 这个算子，理解 这个算子在 基线参考 框架下是如何实现的，结构是什么样子的，现有的测试cases都有哪些。然后看下这个算子的作用/结构/公式，要客观分析，充分理解并验证，不要出错，介绍时应该精炼简明准确易懂，生成一个算子文档。
之后，在当前环境下，即conda [redacted] 下，配置 基线参考 运行环境并且跑通这个 [redacted] 基线参考 算子。
之后，按照 cannbot-skills/ops/pypto-pro-kernel 这个 skill 的流程，按照这个 基线参考 算子的功能与 NPU 流水，将其翻译转化为 对应的 pypto-pro kernel。目标是在精度打通的基础上使得各个 基线参考 默认运行的 cases 都达到但不限于 1.0 倍的 基线参考 的 performance，两边的性能采集方式严格按照 基线参考 官方给出的msprof 采集方式来，额外注意要默认增加 https://gitcode.com/cann/cann-bench 中 对 pypto-pro kernel 采集性能前的 清 L2 冷启动机制。 注意：当前环境下可能有多个agent在做相同的任务，你不要参考他们的产物，互相之间不要影响，就做自己的，可以搞一个单独的工作目录。

## [assistant] msg_156ff4c7c001nbDgM1rpSyP0uy
<step-start> {"snapshot": "4b825dc642cb6eb9a060e54bf8d69288fbee4904", "type": "step-start"}
<details><summary>reasoning</summary>

I need to understand this large task. Let me start by exploring the operator directory and the skill.

Let me break this down:
1. Understand the 基线参考 operator `attention/grouped_matmul_activation_quant` in `/data/[redacted]/ops-transformer`
2. Understand its structure, tests, formula
3. Generate an operator document
4. Configure 基线参考 environment under conda [redacted] and run the [redacted] 基线参考 operator
5. Translate to pypto-pro kernel following the `cannbot-skills/ops/pypto-pro-kernel` skill
6. Target: precision pass + performance >= 1.0x 基线参考 baseline, using official msprof measurement + L2 clear cold start mechanism from cann-bench

Let me start exploring. This is a big task so I'll use a todo list.

Let me first look at the environment and directories.
```

敏感字段已模糊　　1080p / 30 fps

理解需求

源码、环境与验收目标在开场说明

关键节点停留 10 秒 · 需求说明

00:00 / 01:00

重要节点：需求交接

请充分理解 /data/[redacted]/ops-transformer 下的 attention/grouped_matmul_activation_quant 这个算子，理解 这个算子在 基线参考 框架下是如何实现的，结构是什么样子的，现有的测试cases都有哪些。然后看下这个算子的作用/结构/公式，要客观分析，充分理解并验证，不要出错，介绍时应该精炼简明准确易懂，生成一个算子文档。

之后，在当前环境下，即conda [redacted] 下，配置 基线参考 运行环境并且跑通这个 [redacted] 基线参考 算子。

之后，按照 cannbot-skills/ops/pypto-pro-kernel 这个 skill 的流程，按照这个 基线参考 算子的功能与 NPU 流水，将其翻译转化为 对应的 pypto-pro kernel。目标是在精度打通的基础上使得各个 基线参考 默认运行的 cases 都达到但不限于 1.0 倍的 基线参考 的 performance，两边的性能采集方式严格按照 基线参考 官方给出的msprof 采集方式来，额外注意要默认增加 https://gitcode.com/cann/cann-bench 中 对 pypto-pro kernel 采集性能前的 清 L2 冷启动机制。 注意：当前环境下可能有多个agent在做相同的任务，你不要参考他们的产物，互相之间不要影响，就做自己的，可以搞一个单独的工作目录。

transcript.md 第 9 - 11 行

Pro Agent 状态：理解需求（流程示意）



CANNBot LLM自动优化-PANKO：结构与分块联合优化

结构由 LLM 提议，分块由 TPE 搜索，采纳由固定口径的实测决定

1. USER — 2026-10-20 20:47:00 +08:00

让Agent开始使用PANKO优化

- agent=pypto-pro-op-orchestrator use PANKO to optimize /data/[redacted]/panko-MOHV-pyptopro/custom/cannbench/mha/a5/local/case07.py. 100 evaluation budget. you make optimization folder under ./mha

OpenCode view — PANKO on `mha/a5/local/case07.py`

One user prompt; the orchestrator dispatches subagents until PANKO stops. This is the chat-shaped flow: each card is a `task` dispatch and its returned directive. Session `ses_ea6b2d578ffetSStgjPvKD0L8C` · window 2047–0730 (+08:00). Setup and file/NPU exploration are collapsed; raw evidence is `panko_section_main.md` / `panko_section_main_annotated.md`.

优化迭代

User · 20:47

use PANKO to optimize `/data/y00960395/panko-MOHV-pyptopro/custom/cannbench/mha/a5/local/case07.py`. 100 evaluation budget. you make optimization folder under `./mha`

· **setup:** read the PANKO skill; `case07.py` has two JITs (`mha_preload` / `mha_finalize`). · **env:** NPU 5 idle, CANN 9.1 + msprof 9.3, Python 3.13 / optuna 5.0, arch `a5` . · **preopt:** `mha_finalize` 210.6 µs → switched to `mha_preload` 206.3 µs.

21:01 → `pypto-pro-op-autotuner`

PANKO INIT mha case07 └ `CODE` action `u7` — INTERLEAVE N iterations of an outer pypto.loop into an inner one, so N independent chains run concurrently instead of one after another. Replicate eve

21:03 → `pypto-pro-op-coder`

Apply PANKO delta u7 └ Verdict: INEXPRESSIBLE — file left byte-for-byte unchanged** (`sha256 7579c9c1…4723042c`, identical to `optimization/nodes/<preopt_hash>.py`; no git d

21:05 → `pypto-pro-op-autotuner`

PANKO STEP u7 └ `CODE` action `u2` (core-applied) — F-10 Vector tile shapes — core-applied draw (label tpe, trial 3, sites=1), already written to the kernel by bayesian_optimization.apply; do NOT dispat



结果

优化结果

项	数值
目标	`custom/cannbench/mha/a5/local/case07.py` 的 JIT #1 `mha_preload`（共 2 个 JIT，见下）
preopt	206.31 µs
交付 best	**159.41 µs**（J 0.6273，speedup 1.2943x search）
ABBA 确认	**1.2909x**，CI [1.2556, 1.3272]，preopt 206.159 / delivered 159.8065
预算	100 / 100，stop reason `eval_budget`
前沿	6 个非支配点，全部 verified
wall	~9h21m

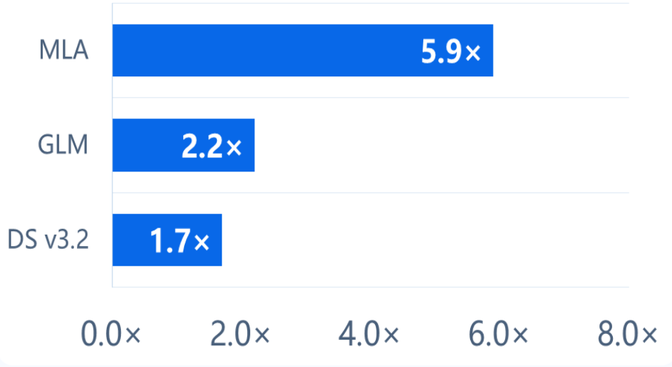
主要优化

真正带来收益的是 **删除 preload 里已经 dead 的 P·V 写回链路**（`mha_finalize` 才是真正写 output 的 kernel）：

- 删除 dead PV writeback：`accumulate/accumulate_1` → `expand_div` → `cast` → `store`
- 删除 36 条不再被读取的 P·V `matmul/matmul_acc`
- 删除 dead `pl.insert(p_1, nzsrc)` 的 UB→L1 拷贝（**最大的一刀，约 −11%**）
- 删除 dead 的 V GM loads（`slot_39/40/41`，`aic_mte2 0.360→0.266`）
- 删除 186 条被后续 window 覆盖的重复 `set_validshape`
- 删除 dead tile 声明/分组与标量簿记，QK 方向做了 unroll/折叠尝试（该轴测量为平坦）
- `mha_finalize` 在交付版本中保持未改（66 的 finalize dead 声明删除未进入 best）

⚠ 注意：这个 1.29x 是 `mha_preload` **单 kernel** 的收益。该文件是双 JIT，`mha_finalize` 仍约 210 µs，所以 A→B 整体大约是 **1.12–1.13x**；PANKO 的 `p` 只能归因一个 op，本次按 `_Z11mha_preload...` 冻结。

历史加速比



MLA Stage 6：8666.7 → 1461.2 µs
GLM Stage 7：176.0 → 80.6 µs
DS v3.2 Stage 7：249.2 → 144.9 µs

停止后独立回归，再交付

独立实验 11/13 获得改善 | **17.6% 中位时延降幅** | 每算子 20 次预算 · 交付复测 100 次

机制：当前 PANKO 实现；结果：原报告 §3.1、§3.2 历史实验。各案例起点不同，不能直接横比。

CANNBot PyPTO Tensor：11/11 完成生成与验证

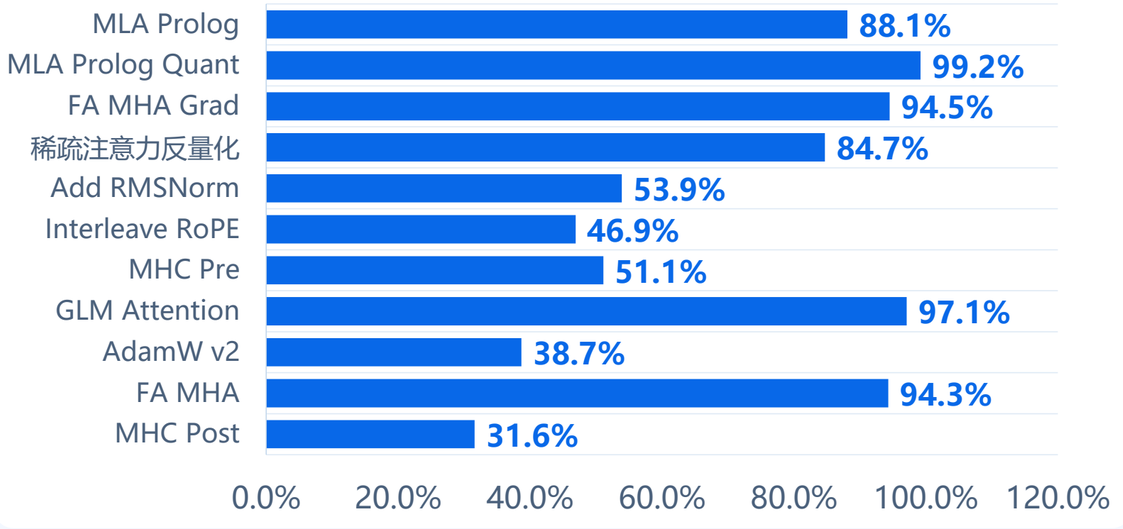
各算子生成投入与时延降幅；降幅均相对本轮生成基线

11/11 一次生成与精度通过 | **11/11 实现审计通过** | 计算主体在 NPU

逐算子生成投入

算子	耗时 / h	Token / M
MLA Prolog	4.49	139.7
MLA Prolog Quant	4.37	139.5
FA MHA Grad	3.82	103.0
稀疏注意力反量化	3.05	88.7
Add RMSNorm	1.10	40.9
Interleave RoPE	1.08	46.1
MHC Pre	1.04	42.9
GLM Attention	0.99	41.8
AdamW v2	0.96	39.7
FA MHA	0.93	38.2
MHC Post	0.86	33.8

调优后时延缩短



生成合计 22.70 h / 754.4 M Token

调优合计 78.19 h / 1,654.8 M Token

来源：0924 轮归档报告（2026-09-28）。Token 含缓存读取；时长为作业合计，结果限于归档配置。

PyPTO Pro Agent：7/8 算子在 2 小时内生成

模型：dsV4.1flash(default) · 覆盖归一化、RoPE、MLA / 量化、稀疏注意力与注意力正反向

0.5–3 h / 算子 | **12.95 h 生成时长合计** | **4.603 亿表列 Token**

8 类算子的生成投入与性能

算子	生成/h	Token/M	主线版本 µs	PyPTO Pro µs	加速比
Add RMSNorm	**0.75**	43.3	230	247.70	92.86%
MLA Prolog	**3**	48.2	87	74.38	**116.97%**
MLA Prolog Quant	**1.2**	74.2	62	46.32	**133.84%**
Interleave RoPE	**0.5**	17.2	8	3.47	**230.30%**
Sparse Attn AQ	**2**	78.1	55	85.93	64.01%
GLM Attention	**2**	48.8	53	49.88	**106.27%**
FA MHA	**2**	74.4	345	435.23	79.27%
FA MHA Grad	**1.5**	76.1	923	1020.08	90.48%

M=百万 Token。生成时长逐项求和，不代表批次历时；Token 统计范围未按生成 / 优化阶段拆分。

4/8 换算加速比 ≥100%　RoPE 最高 230.30%

来源：用户提供的 10/22 表格。Pro 耗时与加速比采用换算口径；加速比=主线版本耗时/未舍入的 Pro 换算耗时。换算值非新增实测。

PyPTO算子编程专场

CANN开发者Meetup

2026.10.24　中国·上海

主办单位： CANN开源社区　**协办单位：** AtomGit



基于 Tensor 模式的 PyPTO 算子开发实践

—— Ling-3.0-flash KDA

分享者：木凡



目录/CONTENT

- PyPTO Tensor 编程
- PyPTO Tensor KDA 算子开发实践

问题

部署一个高效的融合算子，需要哪几步?

01 **算法理解**
设计一套硬件亲和的算法

02 **API规划**
确定需要使用的API

03 **Tiling设计**
数据切块，增加并行度

04 **编写代码**
单算子精度验证通过

05 **整网接入**
验证整网接口

06 **性能调优**
分析流水，优化性能

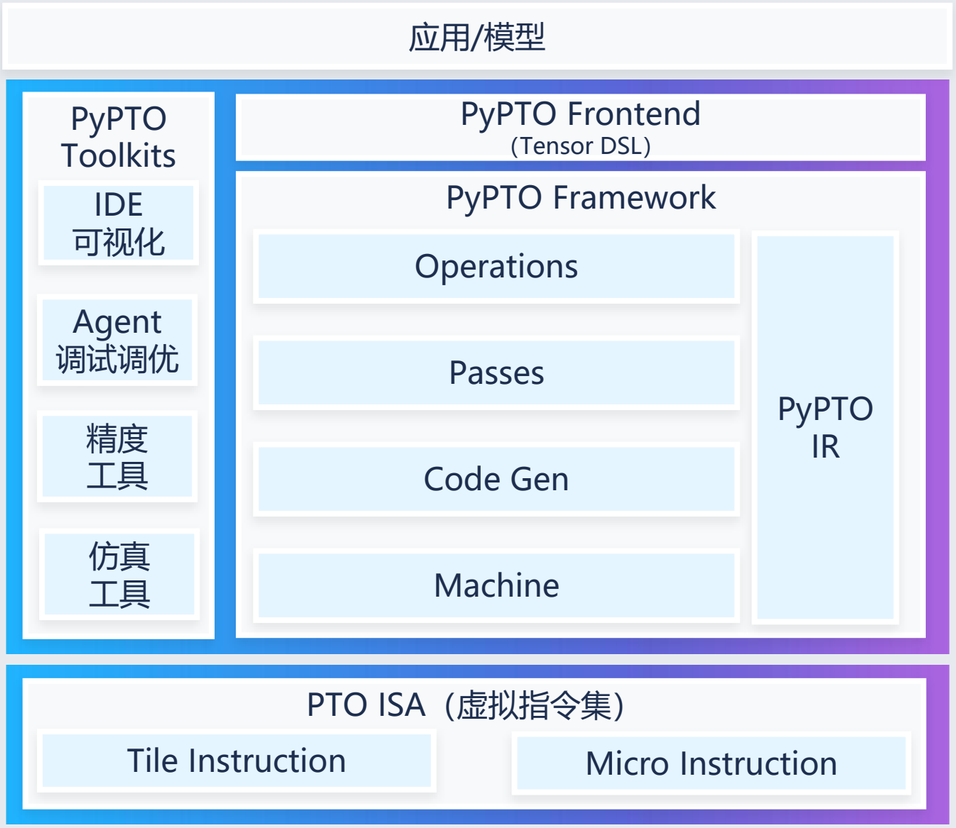
一套包含算子编码在内的完整工作流

难点1：算子编码难度大 → ✓ PyPTO 提供友好的编程框架

难点2：开发流程复杂 → ✓ Agent 编排算子开发流程

CANN

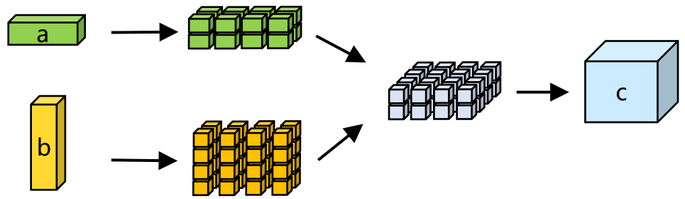
PyPTO：构建简单易用、性能兼顾的算子编程框架



平衡易用性、高性能、通用性，打造极致高效编程体验

自动数据切分

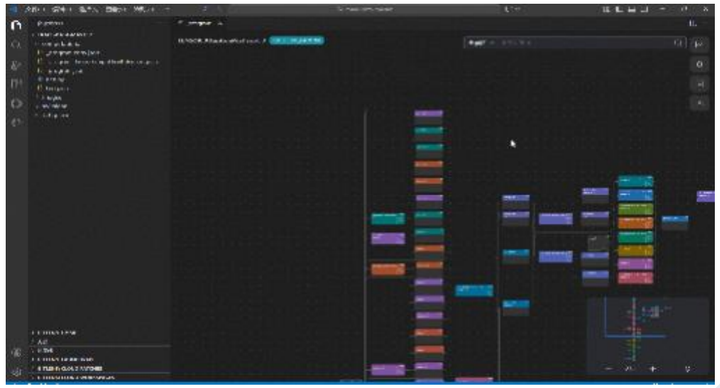
支持自动 Tile 切分，自动适配硬件存储层级
大幅提升编程便捷性



开发者关注于算子 Tensor 计算逻辑及计算流表达，可控制切分粒度，PyPTO 自动实现 Tensor -> Tile 的变换，及运行时数据依赖处理

全调试调优可视化

支持 编译/运行时状态可视化
支持 算子代码-算子计算流程图-算子执行泳道图 实时联动



跨代高效的虚拟指令集底座

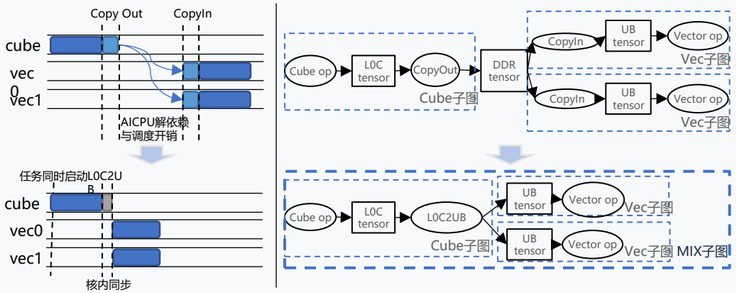
统一指令与内存抽象，封装DSA硬化，实现跨代兼容不降速

随路量化
转置加速
异步预取
……



MIX 子图切分，平滑代际兼容

基于 950 新数据通路，支持自动 MIX 子图切分
大幅提升复杂融合算子性能



Tensor 编程，让复杂算子开发更简单

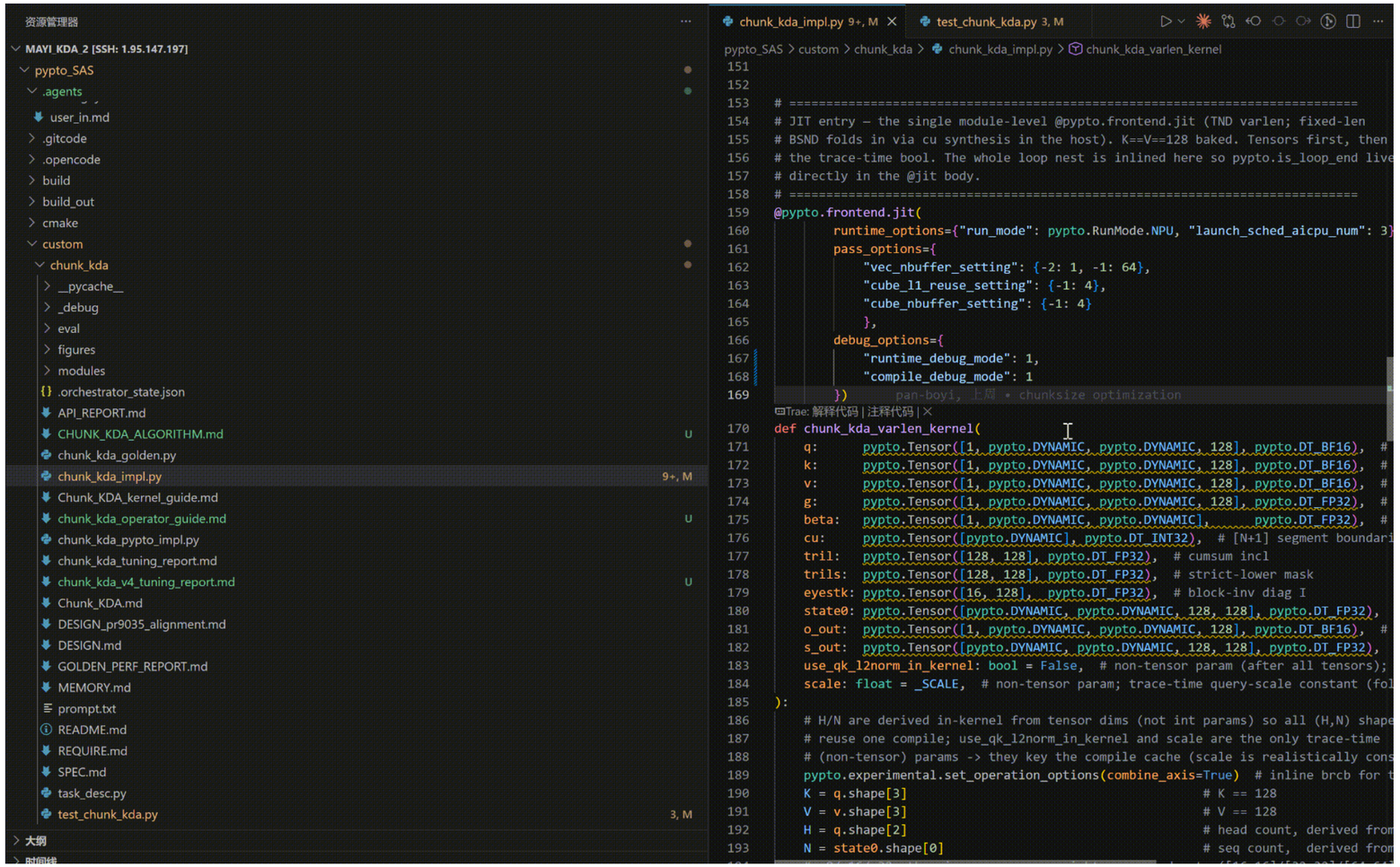
算子描述“做什么”，框架自动处理“怎么做”

- **易开发：**采用 Tensor API，接近算法公式。开发门槛低，代码模块化程度高，耦合度低
- **易迁移：**算法与硬件解耦，减少适配成本，跨代兼容
- **自动 tiling：**算子通过 set_vec_tile_shapes / set_cube_tile_shapes 设置基本块，框架自动切分
- **易调优：**内置调优开关，根据不同场景灵活配置
- **MPMD 高效调度：**不同核可在同一时间执行不同程序，多核并行，自动负载均衡，不需要手排流水



工具

可视化调试调优，支撑开发者深耕框架，自主精调



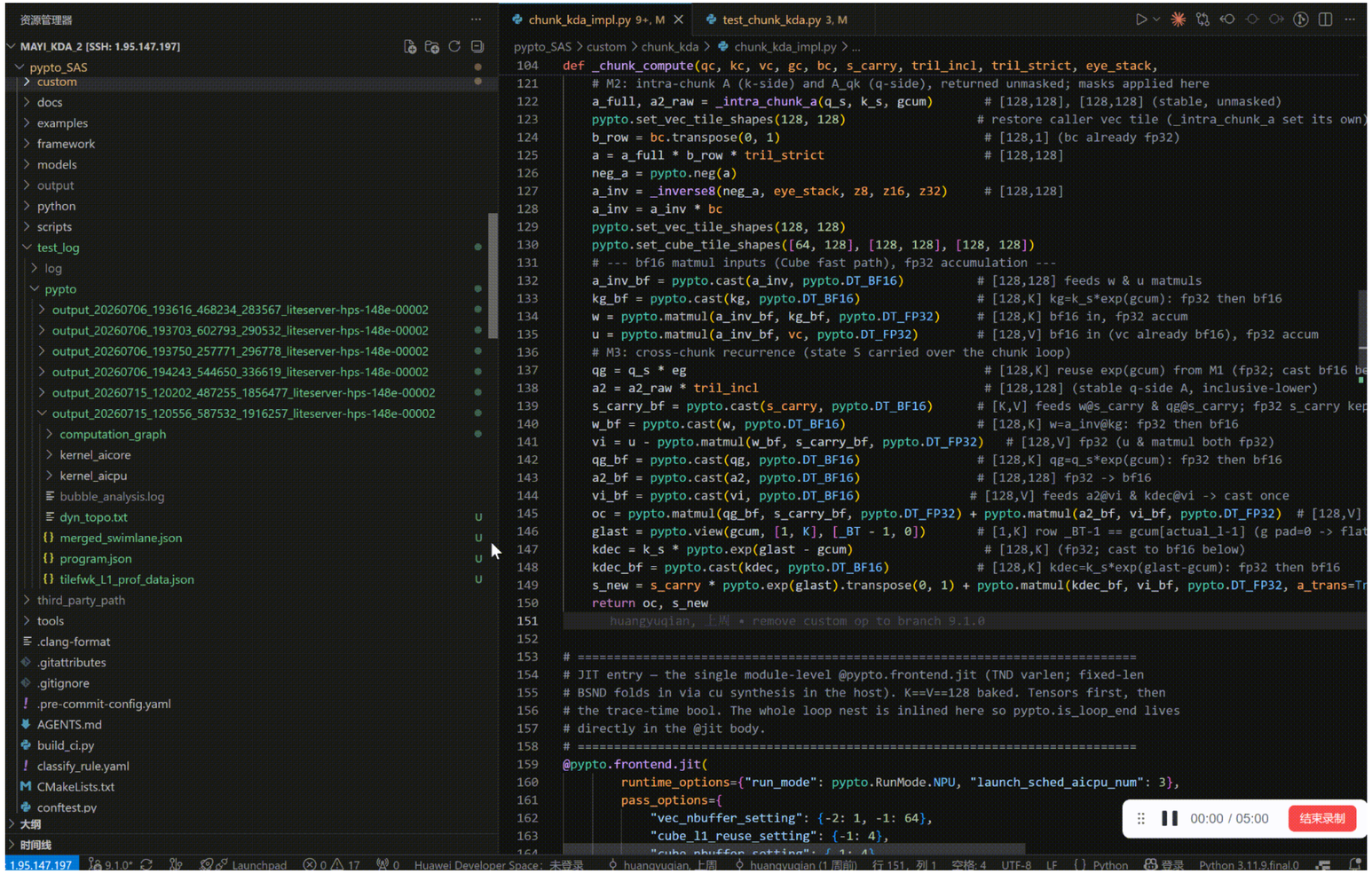
计算图：

- 通过**有向无环图**表示数据流动和计算逻辑
- PyPTO 的计算图包括 Tensor Graph、Tile Graph、Block Graph、Execution Graph，表征了 PyPTO 程序从**抽象计算描述**到**硬件执行**的完整编译流程

Agent 基于计算图（编译产物）进行编译及精度问题定位

工具

可视化调试调优，支撑开发者深耕框架，自主精调



泳道图：

- 提供**核间**、**核内流水**和**统计信息**的可视化展示，支持任务的**依赖关系分析**和多种维度的统计报告信息展示
- 直观展示计算图的**实际调度**与**执行过程**，清晰呈现任务的**执行顺序**和**耗时**信息，帮助开发者**分析算子性能瓶颈**

Agent 基于泳道图（泳道数据）进行性能分析及自动调优

CANN

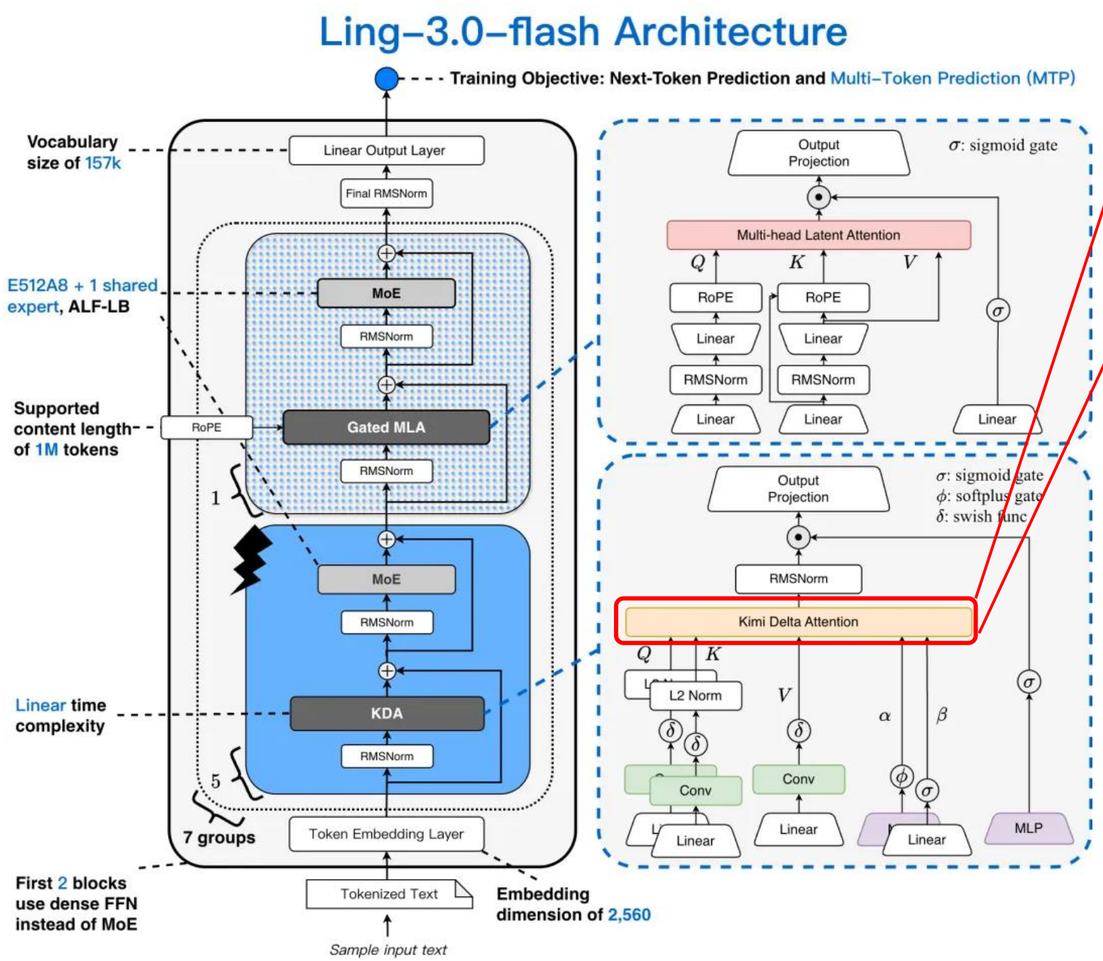
目录/CONTENT

- PyPTO Tensor 编程
- **PyPTO Tensor KDA 算子开发实践**

CANN

模型

Ling-3.0-flash KDA 算子



$$S_t = \left(I - \beta_t k_t k_t^T\right) Diag(\alpha_t) S_{t-1} + \beta_t k_t v_t^T;$$
$$o_t = S_t^T q_t;$$

KDA 与普通 Attention 的区别：

- **线性注意力取代 softmax**：复杂度从$O(L^2)$降到$O(L)$
- **逐通道门控衰减取代标量衰减**：选择性保留和遗忘，表达力更强
- **Delta Rule**：先修正调整，再写入新信息

KDA 难点：

- **计算链路长，一次性编码极易出错**
- **包含矩阵求逆，无原生 API，且求逆计算难以实现并行计算块**
- **块间依赖复杂，串行递推难以表达**

CANNBot 依托内置的 PyPTO 算子专家经验，一周内完成 KDA 算子自动生成、性能调优与整网接入。

CANN

实践

三步完成 PyPTO KDA 算子自动生成

01 需求规划

REQUIRE +SPEC + API_REPORT

02 Golden 参考

PyPTO 友好改写: 对齐 naive_chunk_kda

03 架构设计

分模块、渐进式生成，契约冻结

04 编码构建

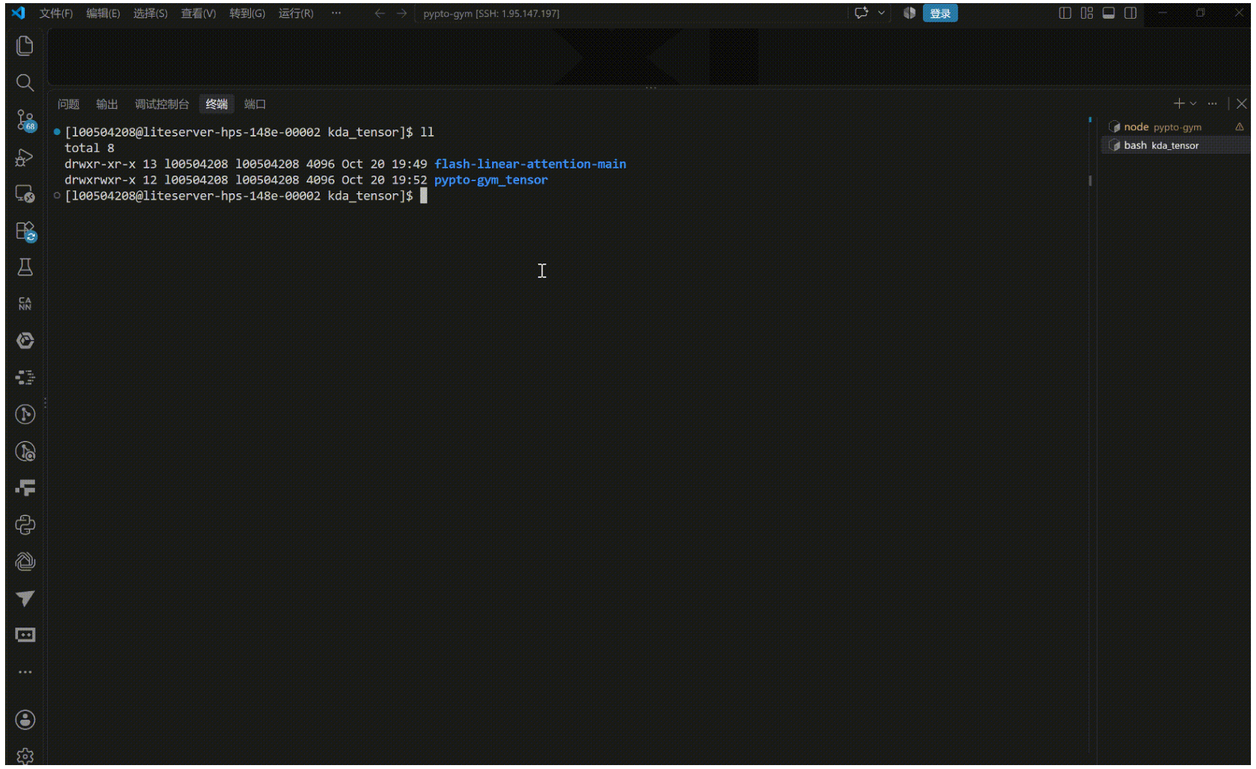
逐模块累积 M1→M12→M123；每 Phase 过精度，M12 一次修复

05 精度校验

测试用例逐一对比；变长 / l2norm / 确定性 / NaN 时序全过

06 性能调优

六轮调优：解串行 + 深流水 + 泳道去碎片 + 合图并行



KDA 算子生成三步曲：

- ✓ **代码仓准备：**Fork pypto-gym仓，并下载到本地
- ✓ **CANNBot 准备：**部署 CANNBot，执行 ```bash cannbot-skills/plugins-official/pypto-op-orchestrator/init.sh project opencode```
- ✓ **kda 算子自动生成：**Agent 交互框中输入Prompt，```请参考flash-linear-attention-main/fla/ops/kda/naive.py这份golden，开发pypto fused_recurrent_kda算子，并进行性能调优```



chunk_kda — Kimi Delta Attention (chunked gated delta-rule linear attention)

带每维遗忘门 g 的 delta-rule 线性注意力，前向算子。按 chunk_size=64 切分序列：chunk 内构造下三角 A 矩阵并用 8×8 分块前向代入求逆完成 delta 修正，chunk 间以状态 S 递推。内部全程 fp32（cumsum/exp/求逆/递推），bf16 输入上游、o 下转 bf16，S 保持 fp32。

算子签名

```
chunk_kda_wrapper(q, k, v, g, beta, scale=None, initial_state=None,
                  output_final_state=False, chunk_size=64) -> (o, S)
```

I/O	变量	Shape	Dtype	说明
入	q, k, g	[B, T, H, K]	bf16	query / key / 每维遗忘门(log 域)
入	v	[B, T, H, V]	bf16	value
入	beta	[B, T, H]	bf16	delta 更新权重
入	scale	scalar	float	None → K**-0.5
入	initial_state	[B, H, K, V] / None	fp32	初始状态 S，None 表零
出	o	[B, T, H, V]	bf16	注意力输出
出	S	[B, H, K, V] / None	fp32	output_final_state=True 返回

支持的 dtype / 形状约束

- dtype：bf16 / fp16 输入（内部 fp32），o 为 bf16，S 为 fp32。
- T 为 64 的倍数，chunk_size 固定 64。
- K == V == 128（P0）；dev 首版支持 K=V=64。
- 动态轴：B 与 H，T 为 64 整数倍。
- bf16 o 容差 rtol/atol 1e-2；fp32 状态 S 容差 1e-3。

最小用例

```
import torch, torch_npu
from chunk_kda_impl import chunk_kda_wrapper

B, T, H, K = 1, 4096, 16, 128
q = torch.randn(B, T, H, K, dtype=torch.bfloat16, device="npu:1") * 0.1
k = torch.randn(B, T, H, K, dtype=torch.bfloat16, device="npu:1") * 0.1
v = torch.randn(B, T, H, K, dtype=torch.bfloat16, device="npu:1") * 0.1
g = torch.nn.functional.logsigmoid(torch.randn(B, T, H, K, device="npu:1")).to(torch.bfloat16)
beta = torch.sigmoid(torch.randn(B, T, H, device="npu:1")).to(torch.bfloat16)

o, S = chunk_kda_wrapper(q, k, v, g, beta, output_final_state=True, chunk_size=64)
# o: [1,4096,16,128] bf16 ; S: [1,16,128,128] fp32
```

chunk_kda 性能调优报告

1. 调优概述

- 算子: chunk_kda ｜文件: chunk_kda_impl.py
- 用例: P0 = test_chunk_kda_l1 (B1/T4096/H16/K=V=128, output_final_state)
- 目标: NPU 提速 2× (24137 → ≤12069 us)
- 实际: 24137 → **18362 us，1.31× (−24%)**；2× 目标未达（差距 1.52×）
- 设备: 910B3 dev1，真实 NPU，无 sim

2. 性能对比

指标	基准	最终	变化
执行时间 us	24137	18362	−24%
核利用率	21.4%	−22%	+1pt
气泡率	47.7%	41.5%	−6pt
负载均衡	49%	−52-68%	+
精度全程 PASS：o max_diff 1.526e-5、S 3.725e-9，all_close。			

3. 已采纳优化

阶段	项	改动	收益	行
FRONTEND	F-9 cube tile	M3/w-u matmul mL0 128→64	−1.8%	108,140
FRONTEND	F-15 combine_axis	tail-1 beta brcb	−1.6%	89
SWIMLANE	vec_nbuffer	{-2:1,-1:64}	−20.6%	167

实践

KDA 开发中遇到了哪些难点?

3 Kimi Delta Attention: Improving Delta Rule with Fine-grained Gating

We propose Kimi Delta Attention (KDA), a new gated linear attention variant that refines GDN's scalar decay by introducing a fine-grained diagonalized gate $\mathrm{Diag}(\boldsymbol{\alpha}_t)$ that enables fine-grained control over memory decay and positional awareness (as discussed in §6.1). We begin by introducing the chunkwise parallelization of KDA, showing how a series of rank-1 matrix transformations can be compressed into a dense representation while maintaining stability under diagonal gating. We then highlight the efficiency gains of KDA over the standard DPLR (*Diagonal-Plus-Low-Rank*) formulation [30, 71].

$$\mathbf{S}_t = \left(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top\right) \mathrm{Diag}\left(\boldsymbol{\alpha}_t\right) \mathbf{S}_{t-1} + \beta_t \boldsymbol{k}_t \boldsymbol{v}_t^\top \in \mathbb{R}^{d_k \times d_v}; \qquad \boldsymbol{o}_t = \mathbf{S}_t^\top \boldsymbol{q}_t \in \mathbb{R}^{d_v} \tag{1}$$



KDA 算子分为 chunk_kda 和 fused_recurrent_kda，分别对应 Prefill 和 Decode 两个阶段。

两者计算粒度与调度方式差异明显，但开发中面临的难点是共性的

01 计算链路长导致一次性编码极易出错

门控前缀和、指数衰减、下三角 A 矩阵构造、分块前向代入求逆、delta 校正（w/u）、跨块状态递推等分段计算，融合范围大、数据依赖复杂。

传统手写模式下，开发者通常一次性写完整个 kernel 再调试，错误定位困难。

02 求逆等非标准计算无原生 API

没有原生的矩阵求逆 API。

传统手写模式下，开发者需自行实现分块前向代入，涉及大量地址计算和搬运指令编排，极易出错。

03 串行状态递推难以拆分调试

chunk_kda 块间以状态 S 串行递推，fused_recurrent_kda 逐 token 串行递推。

传统手写模式下，开发者需要手动编排同步指令、管理跨迭代的数据读写时序，依赖关系表达困难。

实践

PyPTO 结合 Agent 如何逐一解决?

计算链路长导致一次性编码极易出错

PyPTO 框架

编程范式具有清晰的分层结构：

- host wrapper 层：负责 layout 适配和输出预分配，不涉及计算逻辑
- JIT kernel 层：将多段计算融合进单个 kernel
- Loop 层：所有循环和计算主体，框架自动处理地址计算与数据搬运等

三层各司其职，边界清晰

Agent 增强

Agent 设计阶段进行模块拆分，按渐进式生成的方式逐步实现，每个模块独立编码、验证通过后再拼接下一个，一次性编码极易出错的问题得到有效缓解。

求逆等非标准计算无原生 API

PyPTO 框架

计算原语对标torch，使用简单、规范，用原语组合即可表达复杂的计算算法。

```
def _inv_min_length(attn_dim1, eye, row_num, col_num):
    size = col_num // row_num
    cur = attn_dim1[:2, :]
    for i in range(2, row_num, 1):
        row = attn_dim1.view([1, col_num], [i, 0])
        row_exp = row.reshape([size, row_num]).view([size, i], [0, 0]) \
                     .transpose(1, 0).reshape([size * i, 1])
        prod = pypto.sum((row_exp * cur.reshape([size*i, row_num])) \
                         .reshape([i, col_num]), dim=0, keepdim=True)
        cur = pypto.concat([cur, row + prod], dim=0)
    return cur + eye
```

Agent 增强

第一，Skill 体系中沉淀已验证的求逆实现。第二，Agent 基于专家经验进行原语组合，而非盲目试错。

两者叠加，非标准计算的实现从"高危手写"变成"可验证生成"。

串行状态递推难以拆分调试

PyPTO 框架

在 Tensor 层，框架自动追踪数据流的读写关系。在循环层，框架自动插入跨迭代的依赖约束。

两个层面叠加，块间依赖的表达从"手写同步指令"变成"声明依赖语义"，框架自动完成依赖分析和调度编排。

Agent 增强

Agent 基于这两层依赖能力进行算子开发时，只需在代码中声明数据切片和标注串行边界，框架自动处理依赖追踪和调度编排，Agent 无需感知底层同步细节。

实践

调优参数化：Agent 自动完成性能调优

chunk 间串行依赖阻断流水

瓶颈分析

chunk_kda 块间以状态 S 递推，每个 chunk 全量算完才进下一个，跨 chunk 流水被完全阻断。

调优方案

PyPTO 框架提供了 Tensor 级的依赖能力构建，让真正有数据依赖的部分施加串行约束，其余计算允许跨迭代重叠。

循环展开参数unroll_list，控制展开深度，增加子图并行粒度，从而进行并行加速。

```
        # possibly-partial tail
        for s_idx in pypto.loop(0, L, _BT, name="LNT", unroll_list=[4]):
            # SNAP:inside_nt_loop
            off = bos + s_idx                                # absolute token off
```

子图碎片化使调度开销过大

瓶颈分析

chunk_kda 的分块求逆产生大量细粒度子图：2080 个平均 1.9us 的小 matmul，60% task 实例 <2us 属纯调度开销。
调度等待和前序依赖等待各占空闲的一半。

调优方案

PyPTO 提供了合图配置项来减少子图碎片：

- Cube L1 复用、Cube nbuffer，让多个同构 Cube 子图合并复用 L1 数据
- Vector nbuffer 配置 Vector 子图合并粒度，可手动指定子图合并范围。

```
    },
    pass_options={
        "vec_nbuffer_setting": {-2: 1, -1: 32},
        "cube_l1_reuse_setting": {-1: 32},
        "cube_nbuffer_setting": {-1: 8},
    },
```

M=1 小 matmul 无法充分发挥 Cube 算力

瓶颈分析

fused_recurrent_kda 逐 token 递推，每个 token M=1 的 Matmul 计算数据量较少，无法发挥 Cube 算力。

调优方案

将 M=1 的 matmul 实现的广播乘法 + 归约求和迁移至 Vector 核，同时通过子图合并、子图并行等方式，提升合并粒度，减少调度开销。

```
S_buf[:] = pypto.mul(S_buf, gate)  # [V, K] * [1, K] → [V, K]
bk = pypto.mul(beta_i, k_i)
kS_prod = pypto.mul(k_i, S_buf)  # [1,K]*[V,K]→[V,K] (broadcast)
kS = pypto.sum(kS_prod, -1, True)  # [V, 1]
kS = pypto.reshape(kS, [1, V])  # [1, V]
```

实践

调优参数化：Agent 自动完成性能调优






```
@pypto.frontend.jit(
        runtime_options={
            "run_mode": pypto.RunMode.NPU,
            "stitch_function_max_num": 256,
            "device_sched_mode": 1,
            "launch_sched_aicpu_num": 3,
        },
        pass_options={
            "vec_nbuffer_setting": {-2: 1, -1: 32},
            "cube_l1_reuse_setting": {-1: 32},
            "cube_nbuffer_setting": {-1: 8},
        },
        )
def chunk_kda_varlen_kernel(
    q:      pypto.Tensor([1, pypto.DYNAMIC, pypto.DYNAMIC, 128], pypto.DT_BF16),
```

经过参数化调优后，子图合并粒度增加，调度开销明显缓解，并行化程度更高，算子性能提升明显。

CANN

真实案例

一周内完成 KDA 算子生成、整网接入、优化与部署

1 **需求与数学基线**

SPEC · API · Golden

2 **模块实现与精度闭环**

M1 → M12 → M123

3 **整网接口接入**

Shape · dtype · ABI

4 **性能分析与部署**

泳道 · 流水 · 回归

- **PyPTO 降低算子的表达门槛**
- **Agent 压缩从算法到部署的工程路径**

目标：让用户快速实现复杂融合算子的开发及部署
适用场景：推理及训练场景，算子快速开发及适配、已有算子快速迁移、算法验证等

CANN

CANN

THANKS



CANN

基于Pro模式的FlexAttention算子开发

高斯拉



目录 CONTENTS

01 框架：PyPTO Pro 的定位与取舍

PyPTO Pro · 框架封装了什么 · 哪些设计必须自己做（第 03–08 页）

02 实践：FlexAttention算子的 Pro 实现

算法· 设计决策 · 流水 · 性能（第 09–16 页）



CHAPTER 01

框架：PyPTO Pro的定位与取舍

从算子开发维度，**易用性**和**高性能**如何兼顾？

昇腾算子编程5大维度

从“数据怎么分”到“指令怎么填”，一个高性能算子的设计空间可以拆成五个维度。

Tiling 切分

- 多核切分
- 负载均衡
- 核内效率

Buffer 管理

- Buffer分配
- Buffer复用
- Buffer轮转

Pipeline 编排

- CV并行
- 核间同步
- Ring Scalar

Tile块 调度

- 偏移计算
- 指令映射
- 指令效率

VF 融合

- Regbase 指令
- 手动UnRoll
- 自动融合

高性能的复杂融合算子开发，需要同时考虑以上五点

CANN

PyPTO Pro通过合理取舍，
保留易用性的同时获取高性能

基于多年算子开发经验，将不影响性能的指令和写法充分封装，最大化提升易用性；

交给开发者

高性能 – 提升性能上限

- Tilling切分
- Buffer分配
- Buffer复用

直接决定性能上限，而且每个算子都不一样：没有通解，只能由懂它的人来做

框架封装

更易用– 提高编码效率

- Buffer轮转
- Pipeline编排
- Tile块调度
- **VF融合**

属于套路式的代码，每个算子差异不大：存在通用封装手段，可以由编译框架完成，且性能无损

CHAPTER 02

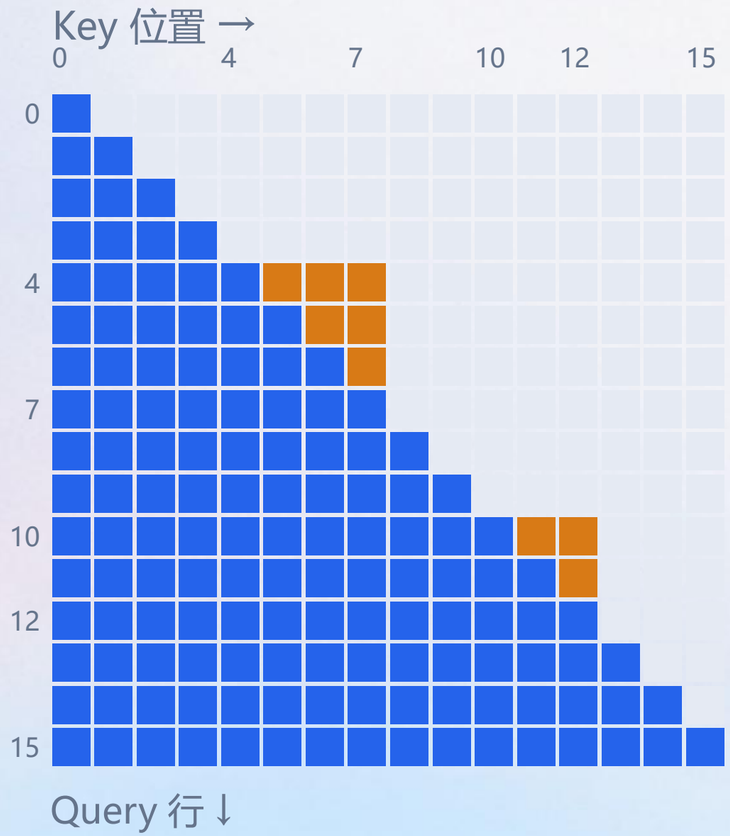
实践：FlexAttention 算子的 Pro 实现

从算法形态到流水排布，逐层拆解一个真实 FlexAttention的落地过程

混合掩码Attention：因果前缀 + 区间内双向可见

公式：O = softmax(QK^T / √d + Mask) × V

落在区间 [s, e] 内的 Query，可以看到该区间的末尾 e。



一个简单规则

q_abs = Skv - Sq + r
bound = max(q_abs + 1, 命中区间的 e + 1)
可见 Key = [0, min(bound, Skv))

图中两段区间

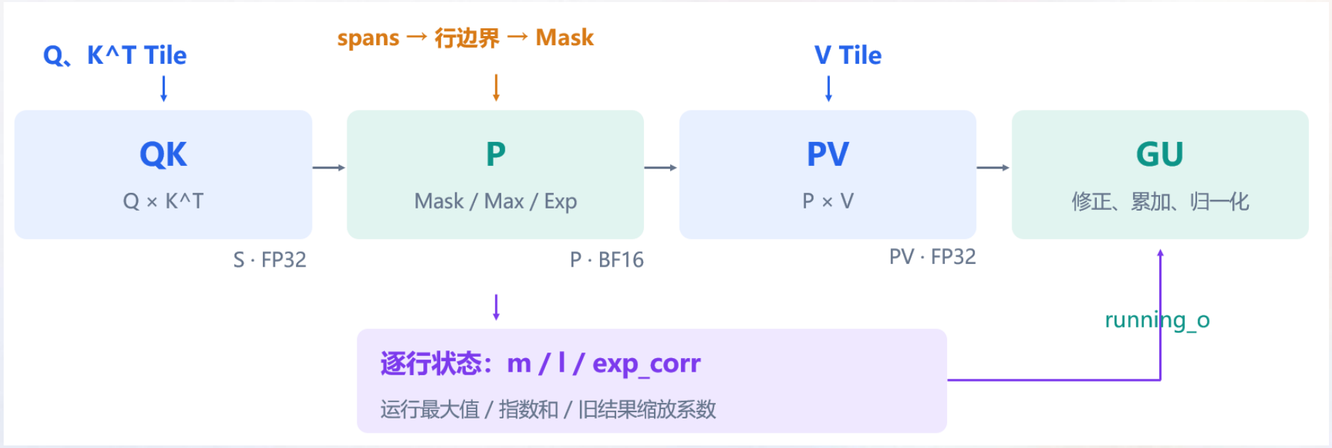
[4, 7]：第 4–7 行的边界都扩展到 8
[10, 12]：第 10–12 行的边界都扩展到 13

因果可见　区间扩展　不可见

每行始终看见一个连续前缀；设备从 spans 生成掩码，Host 无需传完整 Mask 矩阵。

使用PyPTO Pro Agent开发

PyPTO Pro 算子开发变得 Agent Friendly!



人：描述算子

描述公式、计算流、flex mask的处理机制

Agent：算子功能打通、性能0.8x

1. 实现基础精度正确版本
2. 使用自动的流水和同步优化能力提高并行度

根据初版代码，结合开发经验，指导Agent对Vector的VF、P的Buffer数量做优化达成理论性能。

Kernel代码骨架：四个核心Stage

PyPTO Pro 核心特性1：前端描述串行逻辑，框架自动排布CV间流水并插入核间同步。

按数据依赖写主循环，描述串行算子逻辑，用 Tile Group 描述共享的Buffer缓冲。

主循环骨架 · 省略参数与准备代码

```
@pl.jit(arch="3510", auto_mutex=True,
    pipeline=pl.pipeline.PipelineConfig(preload=2))
def flex_attention_bf16(...):
        k_l1_db = pl.make_tile_group(type=pl.TileType(shape=[TD, TKV],
            dtype=pl.DT_BF16, target_memory=Mat, layout=pl.ZN)
        ... #定义各类tile_group
    for i in pl.range(0, n_items):
        kv_end = pl.min(scalar_bound(mm_prefix_range, b_idx,
                    span_num, q_abs0 + s1_size - 1), actual_s2)
        kv_loop = (kv_end + TKV - 1) // TKV # 计算flex边界
        for ki in pl.range(0, kv_loop):
            compute_qk(ctx, k_l1_db,...)
            compute_p(ctx, ...)
            compute_pv(ctx, ...)
            compute_gu(ctx, ...)
```

0 compute_qk — Cube

QK^T → FP32 分数

1 compute_p — Vector / VF

Mask + 在线 Softmax → BF16 P

2 compute_pv — Cube

P × V → FP32 局部输出

3 compute_gu — Vector / VF

历史修正 + 累积 + 最终归一化

Tile = 128 × 128 • preload = 2

Stage 0 | QK：用 Tile API 表达搬运与矩阵乘

PyPTO Pro 核心特性2：pl.load / move / matmul等接口，以Tile为核心表达数据的搬运和计算。
PyPTO Pro 核心特性3：使用tile_group描述buffer轮转，同时自动插入最优的核内同步，简化算子写法。

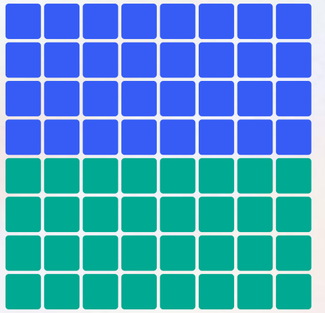
Q 在首个 KV 块加载一次；K 从 page table 定位，GQA 映射到共享 KV head。

compute_qk · 节选，省略 set_validshape 与共享视图轮转

```
k_l1_db = pl.make_tile_group(type=pl.TileType(shape=[TD, TKV],
    dtype=pl.DT_BF16, target_memory=Mat, layout=pl.ZN)
cur_k_slot = k_l1_db.next()
qk_left = left_db.next()
qk_right = right_db.next()
qk_acc = acc_db.next()
if ctx.ki == 0:
    pl.load(cur_q_slot, q, ...)
pl.load(cur_k_slot, k,
    [ctx.kv_page, ctx.kv_slot, ctx.kv_n_idx, 0],
    order=[3, 1])
pl.move(qk_left, cur_q_slot)
pl.move(qk_right, cur_k_slot)
pl.matmul(qk_acc, qk_left, qk_right)
qk_slot = qk_vec_db.next()
pl.move(qk_slot, qk_acc,
    acc_to_vec_mode=pl.AccToVecMode.DualModeSplitM)
```

DualModeSplitM

一个 Cube Tile，两个 Vector 子核



AIV 0
64 × 128

AIV 1
64 × 128

ACC 128 × 128

Stage 1 | P：直接操作Reg来实现Mask与FlashSoftmax

PyPTO Pro 核心特性4：提供手写Regbase来完成Vector侧高性能 & 精细化的操作。

用寄存器级 API 处理reduce max、exp、量化和 NZ 写出等vector操作，实现最高性能。

01 使用Scalar计算KV的上限

通过外部输入 + q的位置计算KV长度的上限

↓

02 使用寄存器指令将KV上限转换成掩码

使用VF将bound实时转化成掩码矩阵

↓

03 Attention Score(ND) → P(NZ)

Scale / Mask / Max / Exp / Sum / Astype/ Nd2NZ

02 使用寄存器加个KV上限转换成掩码

```
@pl.vector_function
def compute_row_bounds_vf(...):
    vreg_qabs = vf.arange(q_abs0, dtype=pl.DT_UINT32)
    vreg_bound = vf.adds(vreg_qabs, 1, preg_all)
    for h in pl.range(0, span_num):
        ... # broadcast span endpoints: vreg_s / vreg_e
        preg_hit = vf.ge(vreg_qabs, vreg_s, preg_all)
        preg_le = vf.le(vreg_qabs, vreg_e, preg_all)
        preg_hit = vf.and_(preg_hit, preg_le, preg_all)
        vreg_e1 = vf.adds(vreg_e, 1, preg_all)
        vreg_big = vf.max(vreg_bound, vreg_e1, preg_all)
        vreg_bound = vf.select(vreg_big, vreg_bound, preg_hit)
    vreg_bound = vf.mins(vreg_bound, skv, preg_all)
    vf.store_align(bound_tile, vreg_bound, preg_all)
```

03 VF FlashSoftmax: Attention Score(ND) → P(NZ)

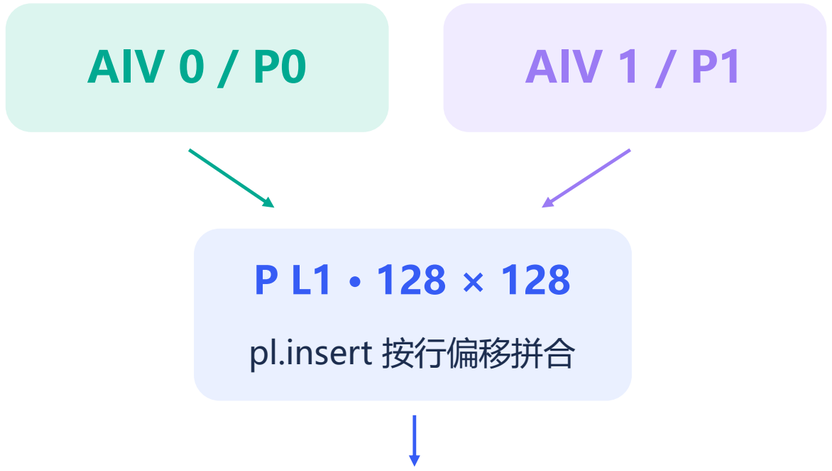
```
@pl.vector_function
def process_vec1_nd_no_update_vf(...):
    ...
    vreg_x = vf.select(vreg_min, vreg_x, preg_mask)
    ... # reduce max + store row max
    vf.mem_bar(mode=pl.MemBarMode.VST_VLD)
    ...
    vreg_exp_even = vf.exp_sub(
        vreg_x_2, vreg_max_2, preg_all)
    ... # reduce sum before BF16 cast
    vreg_exp_even_b16 = vf.astype(
        vreg_exp_even, preg_all,
        layout=pl.CastLayout.ZERO, dtype=pl.DT_BF16)
    ... # pack + store with NZ strides
```

Stage 2 | PV：把两半 P 拼回 L1，再送回 Cube

PyPTO Pro 核心特性5：共享 Tile 通道连接 Cube / Vector；QK与PV使用next()写法复用 L0，无需显式考虑轮询位置。

P 以 BF16 参与矩阵乘；PV 的 FP32 结果再次按 M 拆给两个 Vector 子核。

Vector → L1 → Cube



compute_p → compute_pv · 节选

```
# compute_p: each AIV inserts its half
cur_p_slot = p_mat_db.next()
pl.insert(cur_p_slot, tile_nz, [row_offset, 0])

# compute_pv: Cube consumes the P tile
cur_p_slot = p_mat_db.next()
pl.load(cur_v_slot, v,
    [ctx.kv_page, ctx.kv_slot, ctx.kv_n_idx, 0],
    order=[1, 3])
pl.move(pv_left, cur_p_slot)
pl.move(pv_right, cur_v_slot)
pl.matmul(pv_acc, pv_left, pv_right)
pl.move(pv_slot, pv_acc,
    acc_to_vec_mode=pl.AccToVecMode.DualModeSplitM)
```

PyPTO Pro通过一系列措施，达成Agent Friendly

特性	本例中的落点	开发收益
Tile API	load / move / matmul / insert	按数据块组织搬运与计算
Tile Group + next()	UB、L1上的多缓冲 + 多阶段共享	集中表达缓冲复用，免除核内同步
Stage + pipeline	4个CV Stage描述 + 自动排布流水	只需要描述计算逻辑，免除CV核间同步
Tile + DualModeSplitM	CV配比1:2的数据按Tile粒度拆分	Cube和Vector协作，最大化算力发挥
手写 VF	Mask / Exp / Reduce / BF16 / NZ	精细化的mask操作 + 高性能的FlashSoftmax

通过易用性封装，在推理等性能要求较高的场景，使用Agent可以快速生成高性能融合算子。

性能实测：耗时、流水利用率与 MFU

代码大部分使用**PyPTO Pro Agent**生成，性能达成Vector Bound；MFU 按有效 Attention 模型 FLOPs 计算。

Ascend950 · BF16 / D=128 / page=128 · 28Cube + 56Vector · CANN 9.2

Case 配置	耗时 (μs)	Cube 利用率	MTE2 利用率	MFU*	Vector 利用率	Scalar C 利用率	Scalar V 利用率
C1 长序列 causal B=1 · Sq=Skv=8192 · Hq/Hkv=32/4	1,762.9	86.1%	83.5%	72.2%	95.1%	54.3%	38.0%
C2 多 batch hybrid B=4 · Sq=Skv=8192 · Hq/Hkv=64/8	14,126.7	86.3%	90.3%	74.3%	95.0%	54.3%	38.0%
C3 16K 长上下文 causal B=1 · Sq=Skv=16384 · Hq/Hkv=64/8	13,612.7	86.5%	90.5%	74.8%	94.7%	53.8%	36.9%
C4 深缓存 right-down B=4 · Sq=2048, Skv=16384 · Hq/Hkv=32/4	6,454.6	87.2%	84.2%	75.2%	94.9%	53.8%	36.3%
C5 多请求 hybrid B=8 · Sq=Skv=4096 · Hq/Hkv=32/4	3,597.9	85.7%	84.2%	73.0%	95.5%	55.3%	40.2%

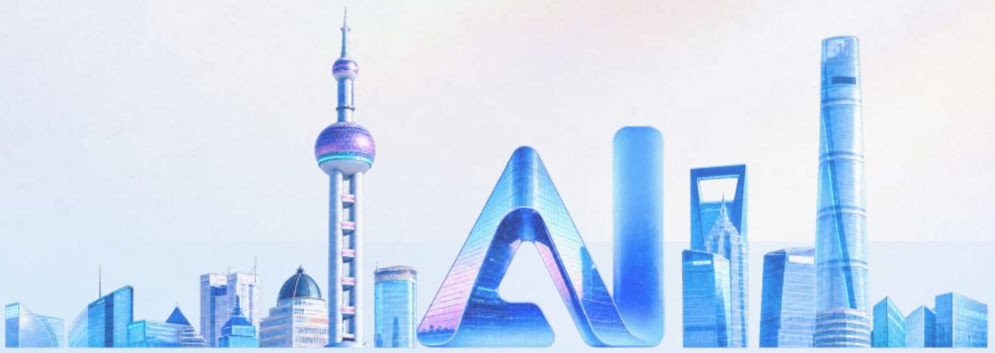
统计：预热5次、3次采样；耗时取中位数，利用率按核时间加权（Cube 28核 / Vector 56子核）。

MFU*：有效 QK/PV FLOPs ÷ (432 TF/s 参考峰值 × 耗时)；Scalar C / V 分别表示 Cube / Vector 侧标量流水。

THANKS

PyPTO Pro

https://atomgit.com/cann/pypto



PyPTO算子编程专场

CANN开发者Meetup

2026.10.24　中国·上海

主办单位： CANN开源社区　**协办单位：** AtomGit



CANN

PyPTO 端侧能力介绍及实践

嘤鸣



目录 CONTENT

■ PyPTO端侧能力介绍

■PyPTO 端侧算子开发实践

麒麟平台广泛支持华为终端产品，针对AI计算范式变革

2017 麒麟970 独立NPU架构

2019 麒麟990 自研NPU

2023 麒麟9000S 芯片回归

2025 Mate80系列 麒麟9030

2026 Mate90系列 **麒麟9050**

产品支持



Mate 10



Mate 30



华为手机



Pad



PC



智能座舱

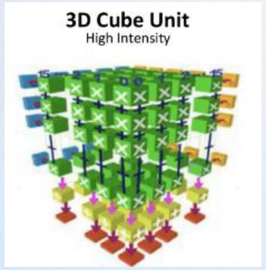
NPU演进

2017 CNN

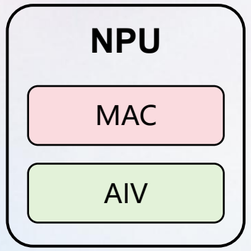
2021 Transformer

2024 LLM

2026 MoE LLM



DaVinci V100



DaVinci V200

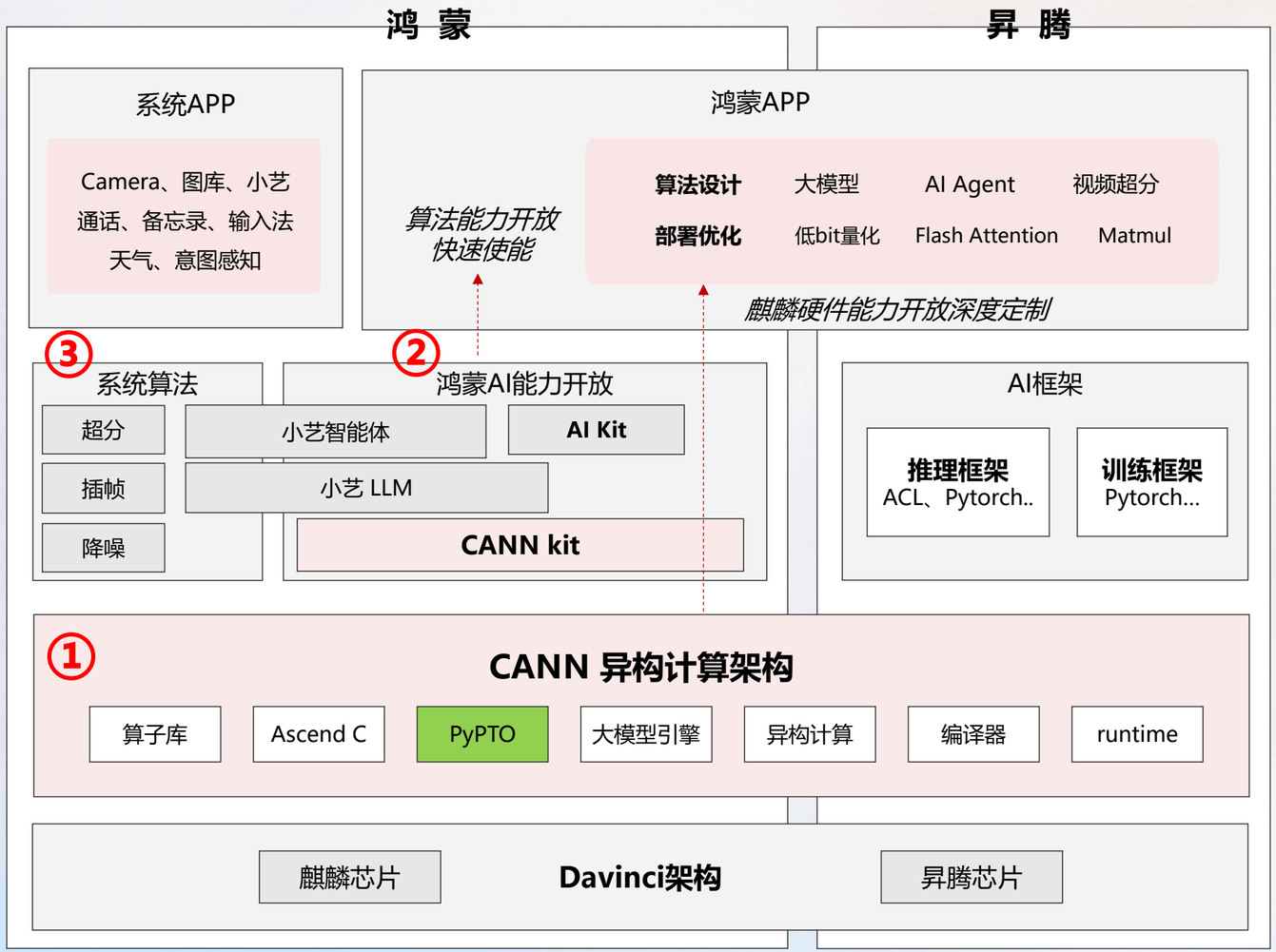
- ✓ 低bit量化计算
- ✓ 带宽优化

DaVinci V300

麒麟/昇腾NPU共架构

CANN

麒麟 AI 战略：基于 CANN 端云协同底座，赋能内外部开发者



① 端云协同底座

- CANN端云算子复用、模型直转、接口复用，生态统一，节约成本
- 案例：QQ音乐、智谱GLM等

② 三方应用智能化

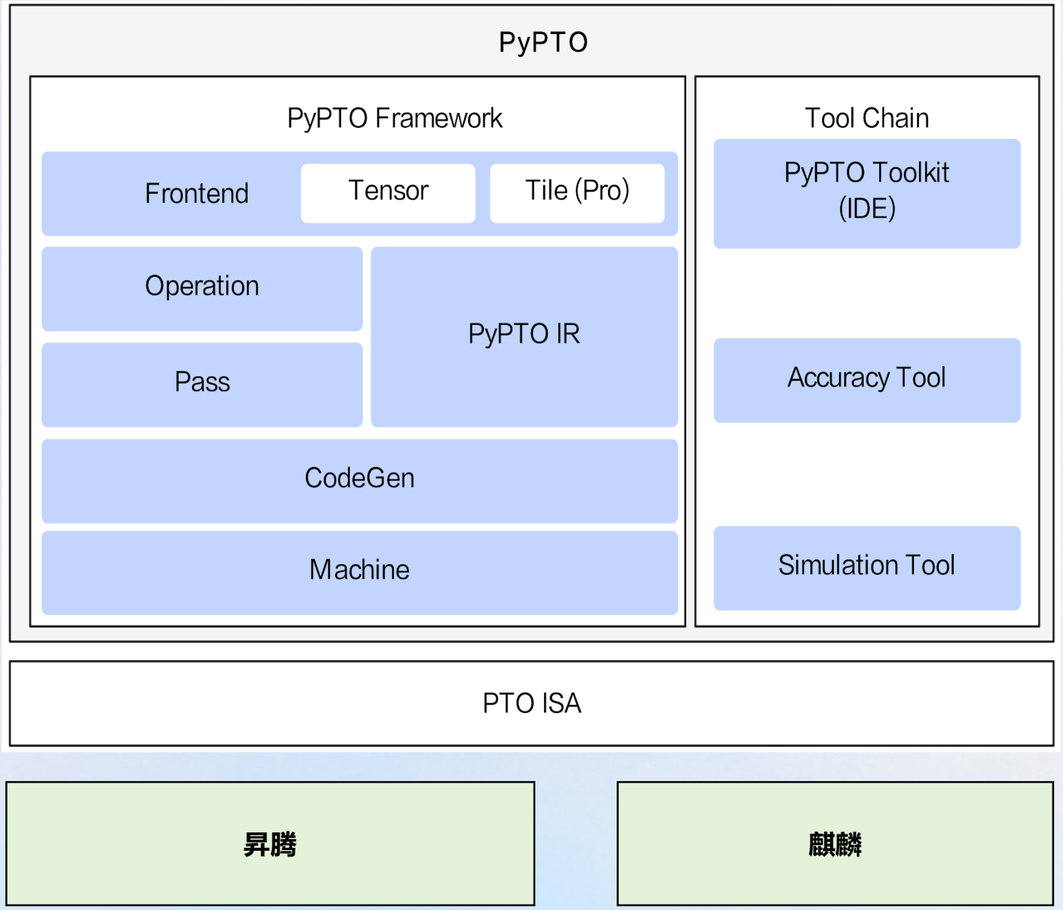
- 鸿蒙AI能力算法、硬件分层开放，支撑各类端侧业务场景
- 案例：悟空图像、咪咕视频等

③ 鸿蒙原生智能

- 软硬芯协同优化，极致能效体验助力鸿蒙竞争力构建
- 案例：小艺LLM、拍照AIRAW/长焦超分等

CANN

PyPTO支持端侧平台



端侧开放能力：

- 和云侧一致的编程接口和工具链能力

支持平台：

- Kirinx90（鸿蒙PC系列）
- Kirin9030（Mate80系列等）
- Kirin9050（Mate90系列等）

支持接口能力：

- Tensor：完全支持
- Pro：规划中
- operation：基础能力已支持，持续丰富中

工具链能力：

- 完全支持

PyPTO算子接入端侧CANN框架

PyPTO开发算子通过离线编译入图方式接入端侧CANN框架

Step1:
识别融合范围

识别通过PyPTO融合优化性能的算子范围

1、**能力补齐**；

2、**性能提升**；

Step2:
PyPTO融合算子开发和验证

pypto前端kernel编写 1 → 编译验证 → 算子精度性能验证

```python
def _pad_branch(x, window, slice_start, length, pad_top, pad_bottom):
    """Slice(axis=2,[start:start+length]) -> per-row window -> constant pad (H -> 256)."""
    n, c, w = x.shape[0], x.shape[1], x.shape[3]
    xs = pypto.view(x, [n, c, length, w], [0, 0, slice_start, 0])
    xm = pypto.mul(xs, window)
    return pypto.pad(xm, [0, 0, pad_top, pad_bottom], "constant", 0.0)


def create_fusion_pad_kernel(shapes, dtypes, soc_version, run_mode=pypto.RunMode.SIM):
    """Factory: builds and returns the fused pad jit kernel.

    shapes/dtypes: 6 inputs — the 3 convtr outputs, then the 3 constant windows [1,1,L,1].
    """
    @pypto.frontend.jit(
        codegen_options={"soc_version": soc_version},
        runtime_options={"run_mode": run_mode},
        debug_options={"compile_debug_mode": 1},
    )
    def fusion_pad_kernel(
        x0: pypto.Tensor(shapes[0], dtypes[0]),
        x1: pypto.Tensor(shapes[1], dtypes[1]),
        x2: pypto.Tensor(shapes[2], dtypes[2]),
        w0: pypto.Tensor(shapes[3], dtypes[3]),
        w1: pypto.Tensor(shapes[4], dtypes[4]),
        w2: pypto.Tensor(shapes[5], dtypes[5]),
        out: pypto.Tensor([...]),
    ):
        pypto.set_vec_tile_shapes(*VEC_TILE)
        p0 = _pad_branch(x0, w0, *BRANCHES[0])
        p1 = _pad_branch(x1, w1, *BRANCHES[1])
        p2 = _pad_branch(x2, w2, *BRANCHES[2])
        out[:] = pypto.add(pypto.add(p0, p1), p2)

    return fusion_pad_kernel
```

```
[DIGITAL_NPU] [rtStreamCreate:25] enter Function>>>
[DIGITAL_NPU] [rtDevBinaryRegister:614] enter Function>>>
[DIGITAL_NPU] [rtMalloc:15] enter Function>>>
[DIGITAL_NPU] [rtMalloc:26] addr:0xaaaae41e1000 size=396
[DIGITAL_NPU] [rtDevBinaryRegister:628] exit Function<<<
[DIGITAL_NPU] [rtFunctionRegister:66] func register funcname kernel_PATH0_hiddenfunc_4_1322159950327261210_main, stubfun 0x1
[DIGITAL_NPU] [rtKernelLaunch:695] enter Function>>>
[DIGITAL_NPU] [rtKernelLaunch:721] Use default threadId 0.
[DIGITAL_NPU] [rtKernelLaunch:732] Task aicache info: en: 0, fm_flag: 0.

[DIGITAL_NPU] [modelSet:555] enter Function>>>
[DIGITAL_NPU] [ModelManager:84] modelSoName: /home/developer/Ascend_0930_aarch64/cann-9.3.0/aarch64-linux/simulator/Kirin9030/lib/libmodel_itf.so
[DIGITAL_NPU] [ModelInit:106] enter Function>>>
[TmSim]: Run in serial mode

[WARNING] pem_log.cc:220 config_aicore_log config.json cannot found, SIM.rotating of log use default value
[error] cfg find path MTE.veccore_ue_mte1 is unqualified.
[error] cfg find path MTE.veccore_ue_fixp is unqualified.
[DIGITAL_NPU] [ModelInit:133] exit Function<<<
[DIGITAL_NPU] [DumpMemInfo:156] memInfo list :5 entries
[DIGITAL_NPU] [DumpMemInfo:159] mem entry 0: addr-0xaaaae3f1ff00, size-24
[DIGITAL_NPU] [DumpMemInfo:159] mem entry 1: addr-0xaaaae41cc000, size-512
[DIGITAL_NPU] [DumpMemInfo:159] mem entry 2: addr-0xaaaae41cd000, size-512
[DIGITAL_NPU] [DumpMemInfo:159] mem entry 3: addr-0xaaaae41ce000, size-512
[DIGITAL_NPU] [DumpMemInfo:159] mem entry 4: addr-0xaaaae41e1000, size-512
[DIGITAL_NPU] [rtKernelLaunch:754] [aicache debug] rtKernelLaunch before ModelTaskRun: aiCacheEn=0, aiCacheFmFlag=0
[DIGITAL_NPU] [ModelTaskRun:160] block start -> block_idx: 0 block_dim: 1 thread_id: 0
tick : 1000[INFO] pem.cc:702 proc_end_pem @1557 Main CCU executing `END` instr.

[TmSim]: time 1658, stopped by sim_stop event
[DIGITAL_NPU] [ModelTaskRun:178] task finish total tick: 1658
[DIGITAL_NPU] [modelUnSet:599] enter Function>>>
[DIGITAL_NPU] [ModelDestory:139] enter Function>>>
[DIGITAL_NPU] [ModelDestory:142] exit Function<<<
[DIGITAL_NPU] [rtMemcpy:44] enter Function>>>
[DIGITAL_NPU] [rtFree:56] enter Function>>>
[DIGITAL_NPU] [rtMemcpy:44] enter Function>>>
[DIGITAL_NPU] [rtFree:56] enter Function>>>
[DIGITAL_NPU] [rtMemcpy:44] enter Function>>>
[DIGITAL_NPU] [rtFree:56] enter Function>>>
[DIGITAL_NPU] [rtStreamDestroy:36] enter Function>>>
INFO:kirin.common:max diff: 0.0

---------------------------------------------------------------------------------------- live log call ----------------------------------------------------------------------------------------
INFO     kirin.common:common.py:22 max diff: 0.0
INFO:kirin.common:max diff index = 0, intput_1 value: -0.535386323928833, intput_2 value: -0.535386323928833
INFO     kirin.common:common.py:24 max diff index = 0, intput_1 value: -0.535386323928833, intput_2 value: -0.535386323928833
INFO:kirin.common:average diff: 0.0
INFO     kirin.common:common.py:25 average diff: 0.0
INFO:kirin.common:cosine similarity: 1.0000000000000002
INFO     kirin.common:common.py:35 cosine similarity: 1.0000000000000002
PASSED

=========================================================================== 1 passed in 3.49s ===========================================================================
```

CANN

PyPTO算子接入端侧CANN框架

PyPTO开发算子通过离线编译入图方式接入端侧CANN框架

Step3:
离线模型生成和上板验证

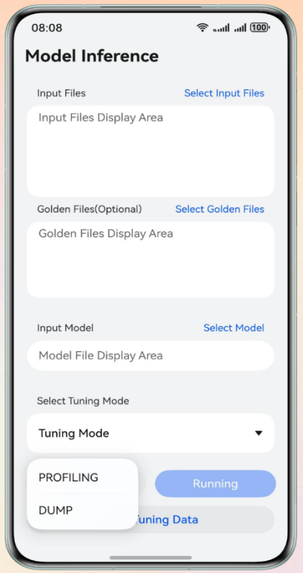
pypto前端kernel编写 1

pytorch模型脚本 → pypto.export → 带pypto的onnx/air模型 → ATC模型转换工具 → 端侧离线模型

PyPTO 入图开发 2

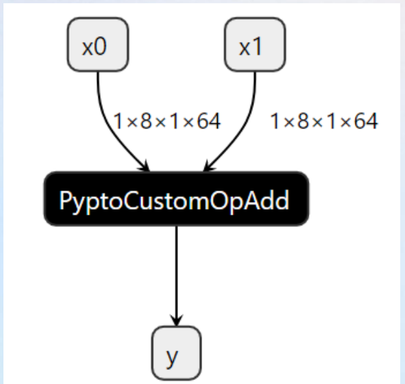
端侧离线模型 → NPUProfiling工具验证整网精度性能

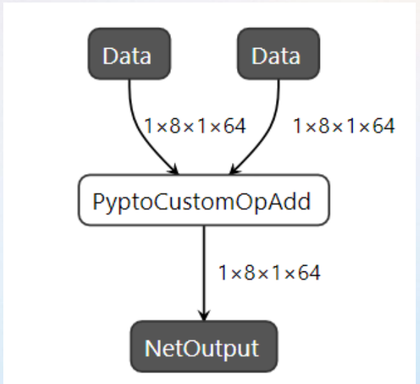
端侧离线模型 → 通过CANNKit接口集成到业务流程中

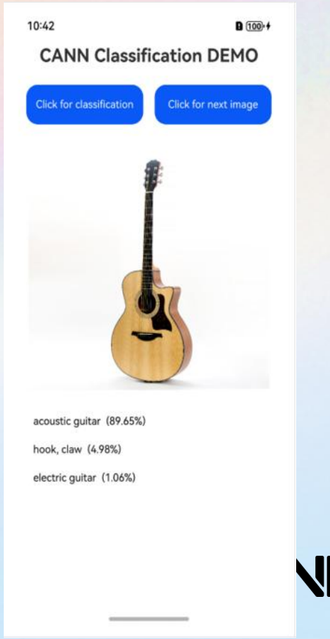



```python
def export_demo(model, path: str):
    pypto.set_codegen_options(support_dynamic_aligned=True)
    input_data0 = torch.rand(SHAPE, dtype=torch.float16)
    input_data1 = torch.rand(SHAPE, dtype=torch.float16)

    out_dir = os.path.dirname(path)
    if out_dir:
        os.makedirs(out_dir, exist_ok=True)
    # base ai.onnx opset to export with
    opset = resolve_pypto_export_opset(model=model, example_inputs=(input_data0, input_data1))
    # register the custom ops (safe to call more than once), then
    # validate/save the model on exit.
    with onnx_export_session(path):
        torch.onnx.export(
            model, (input_data0, input_data1), path,
            input_names=["x0", "x1"], output_names=["y"],
            opset_version=opset, do_constant_folding=False, dynamo=False, report=True,
        )
```







目录 CONTENT

■ PyPTO端侧能力介绍

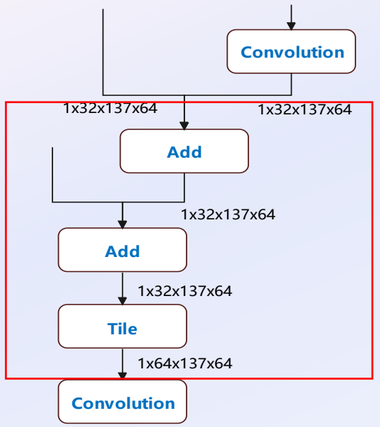
■PyPTO 端侧算子开发实践

业务模型PyPTO优化实践 —— 融合分析

k_model.onnx

通过ATC转为k_model.omc
并通过NPU_Profiling获取
性能结果并分析

case1

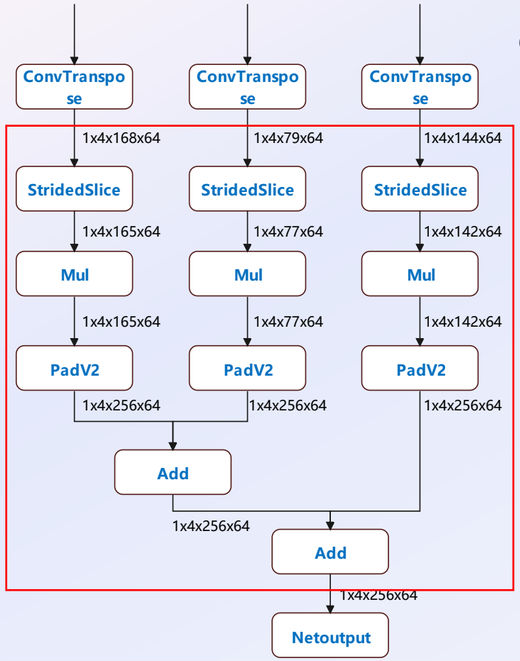


Add+Tile算子耗时：2120 us

整网：10112us

算子占整网耗时21%，期望通过PyPTO融合优化算子性能

case2

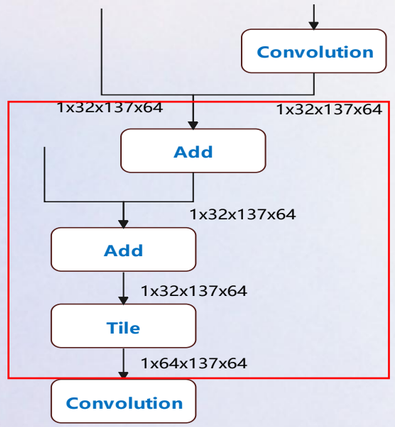


红框内小算子性能：229 us

每个小算子都是memory bound情况，适合通过融合优化性能

CANN

业务模型PyPTO优化实践 —— 融合验证




```python
def create_add_concat_kernel(shapes, dtypes, soc_version, run_mode=pypto.RunMode.SIM):
    """Factory: 3 data inputs [1,32,H,W] -> out [1,64,H,W]."""

    @pypto.frontend.jit(
        codegen_options={"soc_version": soc_version},
        runtime_options={"run_mode": run_mode},
        debug_options={"compile_debug_mode": 1},
    )
    def add_concat_kernel(
        a: pypto.Tensor(shapes[0], dtypes[0]),
        b: pypto.Tensor(shapes[1], dtypes[1]),
        c: pypto.Tensor(shapes[2], dtypes[2]),
        out: pypto.Tensor([...]),
    ):
        pypto.set_vec_tile_shapes(*VEC_TILE)
        x = pypto.add(pypto.add(a, b), c)
        out[:] = pypto.concat([x, x], 1)

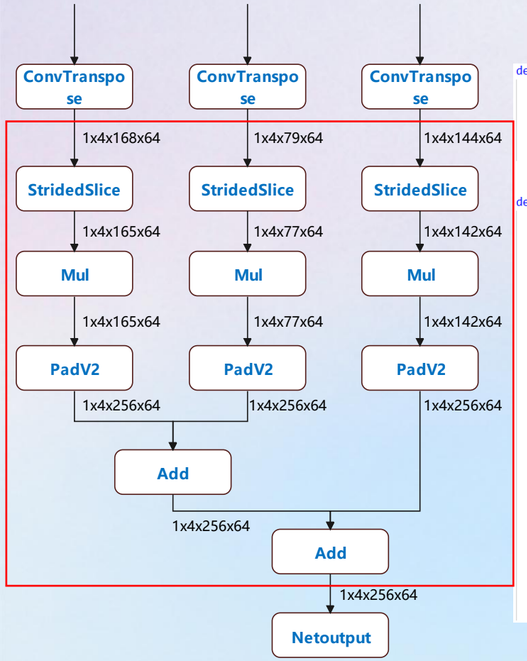
    return add_concat_kernel
```

vector小算子，全部memory bound，原实现性能不足，**通过融合，减少算子间数据访问，优化算子性能。**

融合前小算子性能：2120 us

融合后算子性能： 160 us

融合算子性能优化13倍。




```python
def _pad_branch(x, window, slice_start, length, pad_top, pad_bottom):
    """Slice(axis=2,[start:start+length]) -> per-row window -> constant pad (H -> 256)."""
    n, c, w = x.shape[0], x.shape[1], x.shape[3]
    xs = pypto.view(x, [n, c, length, w], [0, 0, slice_start, 0])
    xm = pypto.mul(xs, window)
    return pypto.pad(xm, [0, 0, pad_top, pad_bottom], "constant", 0.0)


def create_fusion_pad_kernel(shapes, dtypes, soc_version, run_mode=pypto.RunMode.SIM):
    """Factory: builds and returns the fused pad jit kernel.

    shapes/dtypes: 6 inputs — the 3 convtr outputs, then the 3 constant windows [1,1,L,1].
    """
    @pypto.frontend.jit(
        codegen_options={"soc_version": soc_version},
        runtime_options={"run_mode": run_mode},
        debug_options={"compile_debug_mode": 1},
    )
    def fusion_pad_kernel(
        x0: pypto.Tensor(shapes[0], dtypes[0]),
        x1: pypto.Tensor(shapes[1], dtypes[1]),
        x2: pypto.Tensor(shapes[2], dtypes[2]),
        w0: pypto.Tensor(shapes[3], dtypes[3]),
        w1: pypto.Tensor(shapes[4], dtypes[4]),
        w2: pypto.Tensor(shapes[5], dtypes[5]),
        out: pypto.Tensor([...]),
    ):
        pypto.set_vec_tile_shapes(*VEC_TILE)
        p0 = _pad_branch(x0, w0, *BRANCHES[0])
        p1 = _pad_branch(x1, w1, *BRANCHES[1])
        p2 = _pad_branch(x2, w2, *BRANCHES[2])
        out[:] = pypto.add(pypto.add(p0, p1), p2)

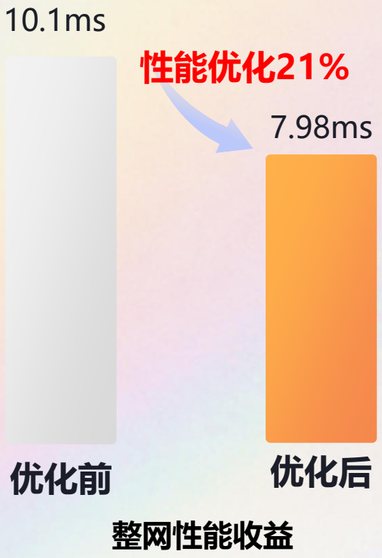
    return fusion_pad_kernel
```

vector小算子，全部memory bound，通过融合，优化算子性能。

融合前小算子性能：228.4 us

融合后算子性能： 56.6 us

融合算子性能优化4倍。



CANN

CANN

THANKS



CANN

PyPTO算子编程专场

CANN开发者Meetup

2026.10.24　中国·上海

主办单位： CANN开源社区　**协办单位：** AtomGit



CANN

PTO ISA：面向昇腾的硬件抽象与跨代兼容

乱序摸鱼



AI编程：三个形态，两次转变

1 逐元素ElementWise

CUDA 出现，成为第一种面向 AI 的编程语言。

2007 — CUDA

2023 — AscendC

性能好，但难写、难规模化。

2 Tensor / Graph编程

框架带来生产力，让开发者描述整个张量。

2015 — TensorFlow

2016 — PyTorch Eager

2018 — JAX

使用框架内置算子集，但失去了对硬件的控制。

3 Tile-Level编程

折中路径：既有生产力，也能感知硬件。

2021 — ★Triton (breakthrough)

2023 — TileLang

2025 — cuTile

2025 — PyPTO

系统层面在生产力与性能之间取舍。

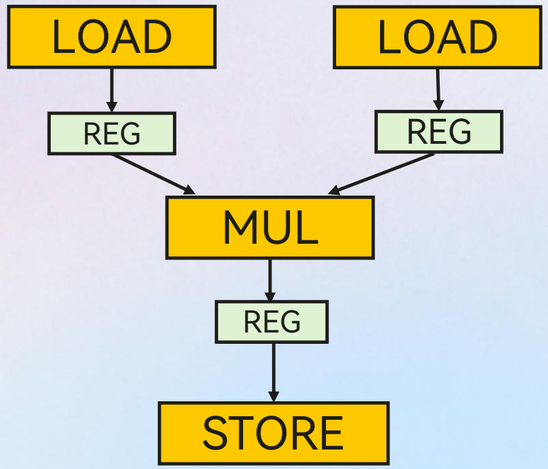
CANN

PTO虚拟指令集： 标量 vs 矢量 vs Tile指令

标量指令集

操作码	寄存器	寄存器

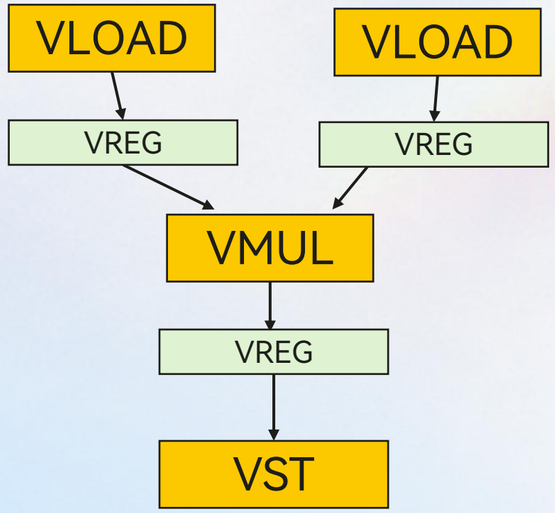
```
load %memref0 -> %a
load %memref1 -> %b
mul  %a, %b -> %c
store  %c, %memref2
```



向量指令集

操作码	向量寄存器	向量寄存器

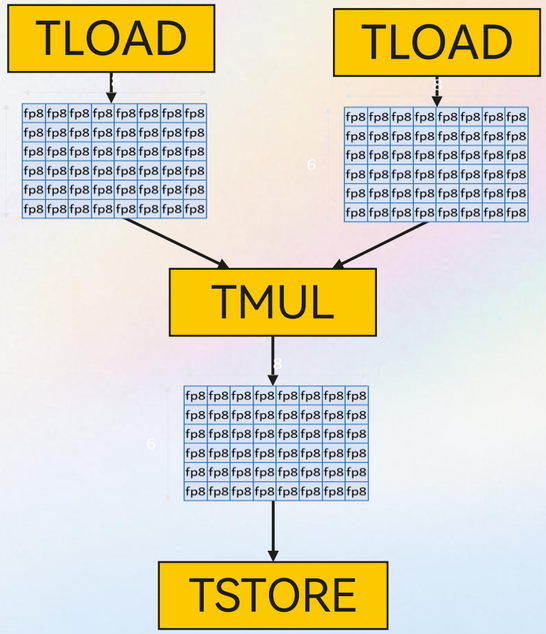
```
vload %memref0 -> %va
vload %memref1 -> %vb
vmul  %va, %vb -> %vc
vstore  %vc, %memref2
```



Tile 指令集

操作码	TILE	TILE

```
pto.tload %tensorA -> %ta
pto.tload %tensorB -> %tb
pto.tmul %ta, %tb -> %tc
pto.tstore %tc, %tensorC
```



PTO-ISA提出的背景：跨代际兼容

挑战：PTO 兼容新旧核架构， 需要“兼容不降速”

- 基于已有PTO指令接口，做到微架构高度适配
- 增强PTO指令约束，满足新架构PPA要求

挑战：算法变动快，PTO需要兼顾易用性和高性能

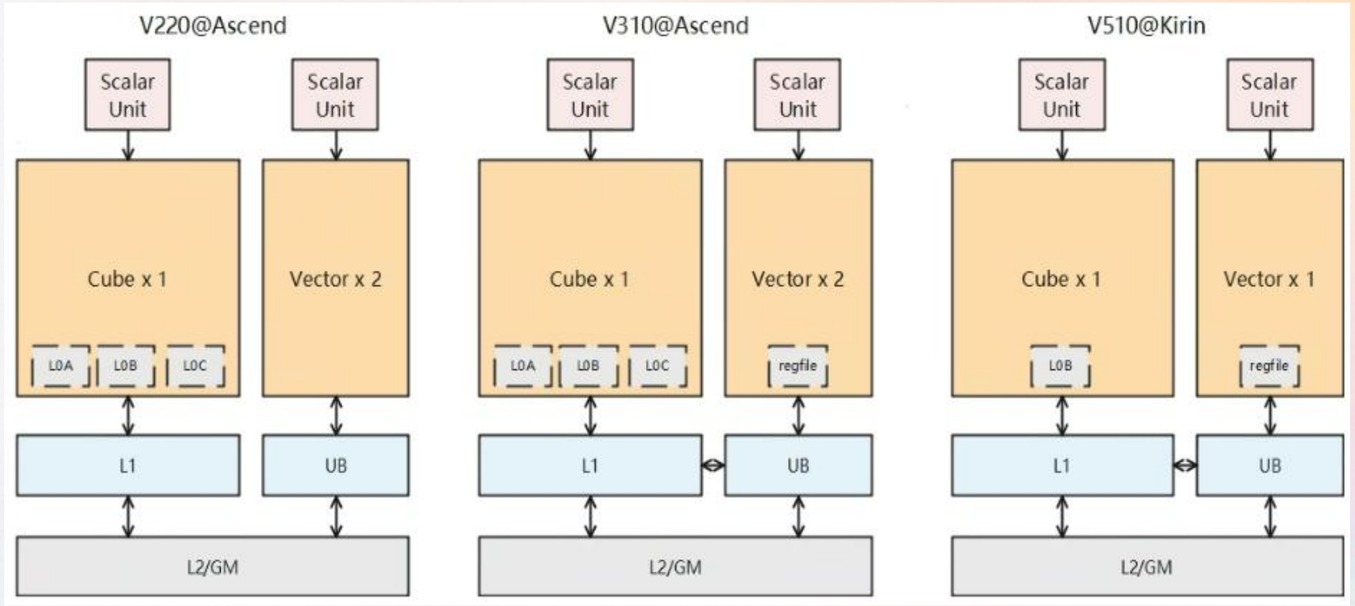
- 提供更方便的排流水的优化
- Profiling提供最够信息更方便融合和软件优化

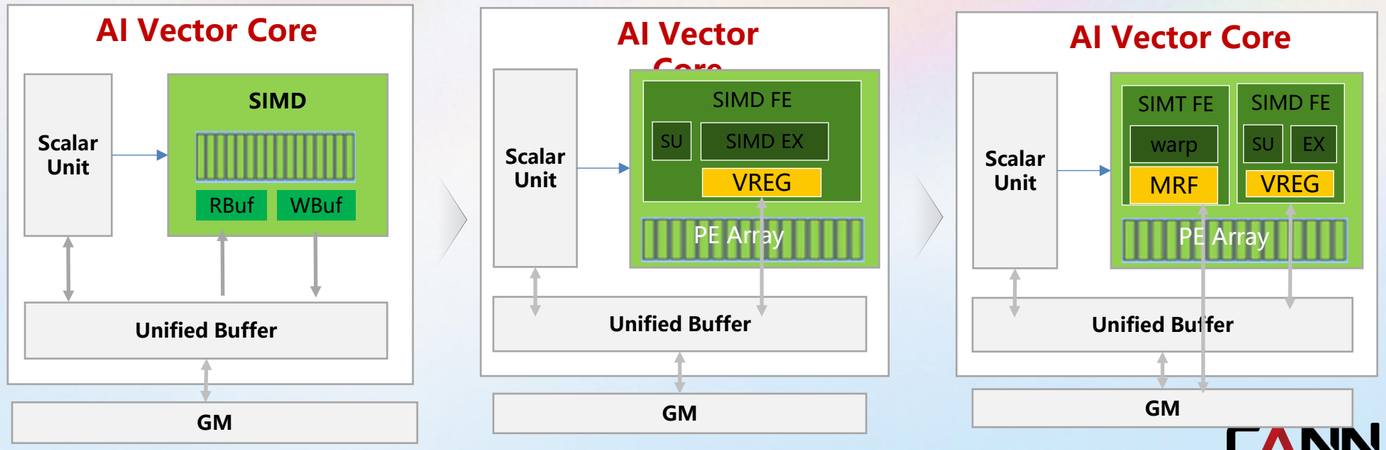
挑战：软硬协同深水区，PTO要封装DSA定制

- 在PTO的约束下，进行有限度DSA硬化
- 承诺不改变访存层级，不改变Memory Model

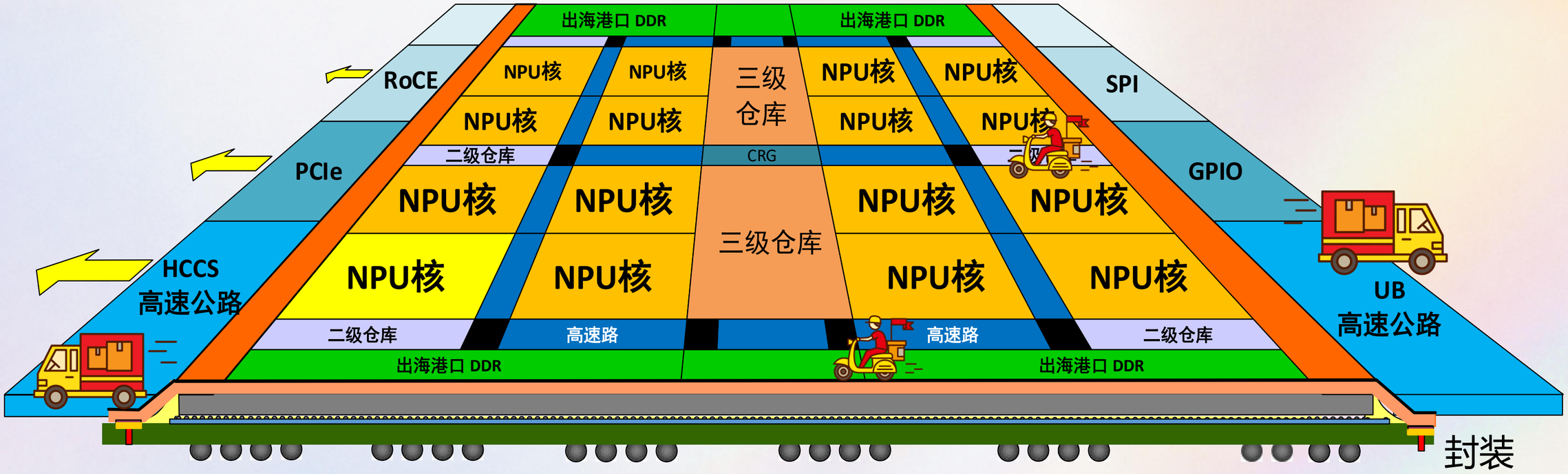
挑战：开放接口，吸引生态使用

- 主动对接TileLang，PyPTO等新语言
- 持续改进，提升易用性





Ascend 950PR硬件介绍



- 32个Cube 64个Vector 工作频率 1.65GHz
- HBM: 128GB 1.6TB/s
- IO: 互联带宽：2016GB/s
- TDP： 630W

- 矩阵乘算力
  - FP32： 27 TFLOPS
  - FP16: 432 TFLOPS
  - FP8/FP6: 864 TFLOPS
  - FP4: 1728 TFLOPS

- 向量算力
  - FP32： 27 TFLOPS
  - FP16: 54 TFLOPS

COMPILATION FLOWS

PTO ISA双重编译流程

A 高级编译流程

High-Level Compile

01 **PyPTO源代码**
TileLang / PyPTO / C/C++

➡

02 **.pto 文件**
PTO IR 文本

➡

03 **ptoas**
PTO 汇编器与优化器

➡

04 **PTO Tile Library**
调用 PTO-ISA C++ 内核函数

➡

05 **bisheng**
编译为目标二进制

✓ 解析、验证、优化 PTO 指令

✓ 将 PTO 指令转换为 C++ 调用

✓ 插入同步（自动同步模式）

B 直接编译流程

Direct Compile

适用于手写显式优化场景，开发者直接使用 PTO-ISA C++ API 编写算子，通过 bisheng 编译器生成目标二进制。

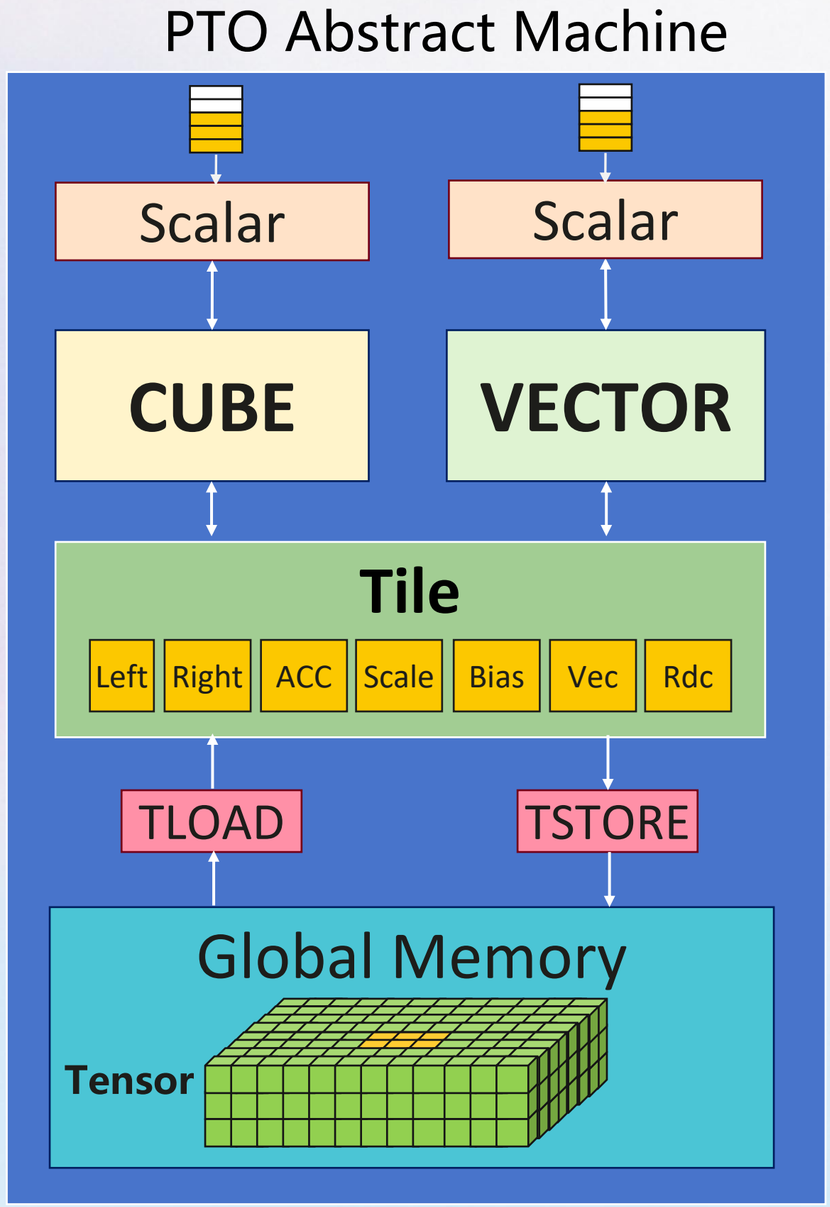
✓ 适合高性能算子开发人员

= 共享语义

两种编译路径共享相同的 PTO 指令语义，仅在最终二进制生成方式上有所不同。

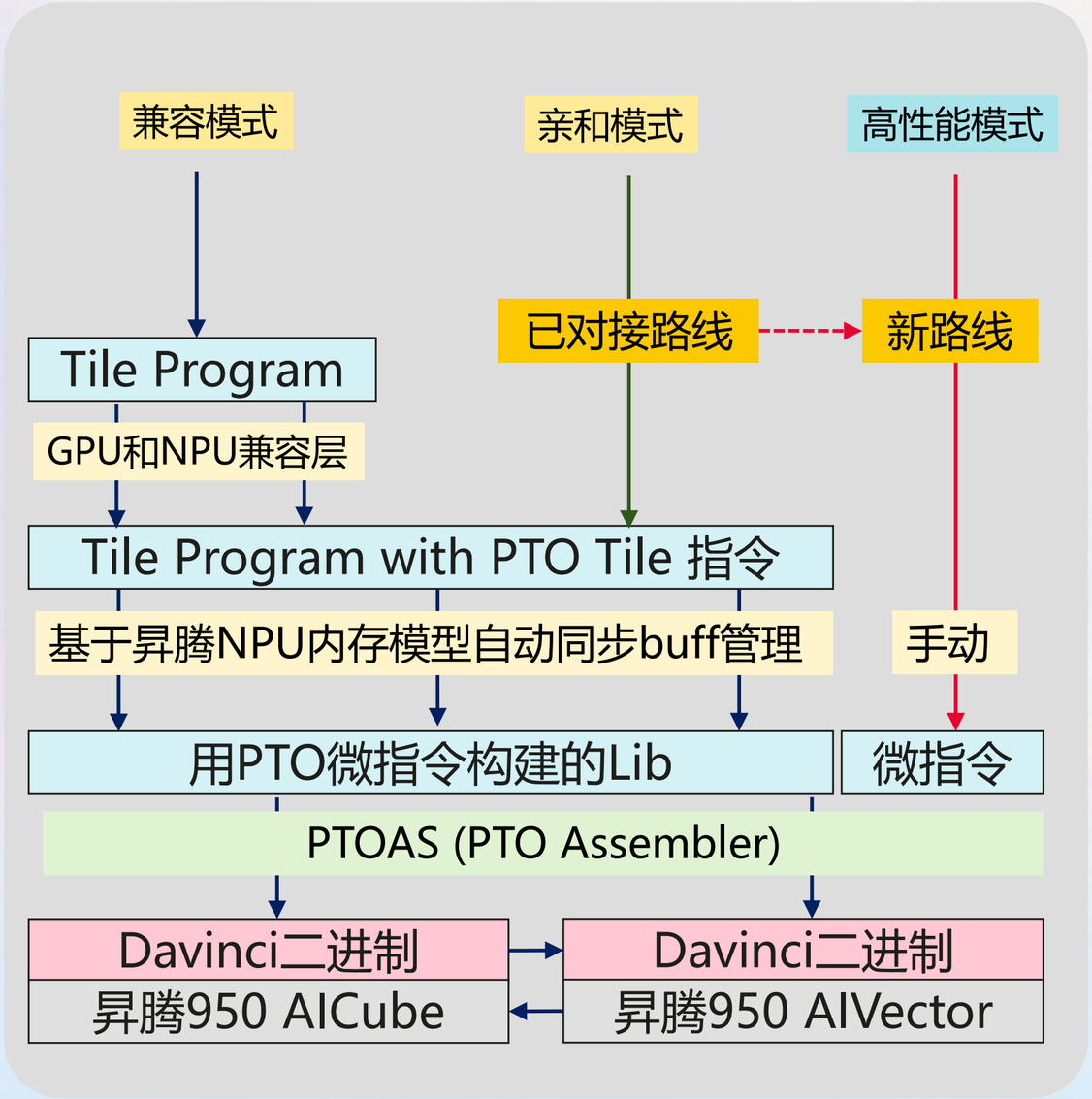
✓ 相同的 Tile 操作语义
✓ 相同的内存模型
✓ 相同的同步机制

PTO ISA Abstract Machine



- **pto.tload** 从GM加载到Mat Tile (L1 Buffer)
- **pto.textract** 从L1抽出来更小的块到Left-L0A, Right-L0B
- **pto.matmul** 从L0A，L0B上做矩阵乘，写到ACC-L0C
- **pto.matmul_acc** 从L0A，L0B上做矩阵乘，累加到ACC-L0C
- **pto.tstore** 从L0C写回GM，并做随路计算

Tile编程框架对接PTO ISA块指令和微指令



PTO ISA在Tile指令基础上开放微指令编程接口

- 为Expert提供手写VF，手动管理Buffer和插同步的能力
- 提供在高性能模式下贴近底层硬件，提供极致性能

PTO ISA为高性能模式下贴近底层硬件，提供极致性能

- 80%场景下依然基于Tile编程
- 20%场景下打开硬件细节启动手动微指令编程

PTO ISA提供PTOAS汇编器 从PTO指令直接翻译到达芬奇二进制

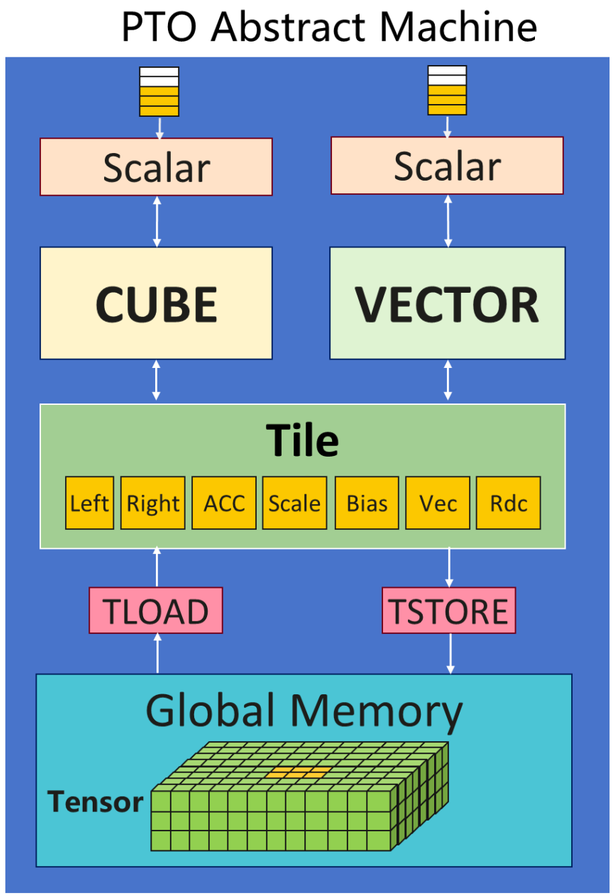
- 提供PTO ISA->BIN的直通路线
- 和联创交互开发，快速识别解决问题，迭代速度快

PTO ISA为联创团队提供底层硬件编程的支持

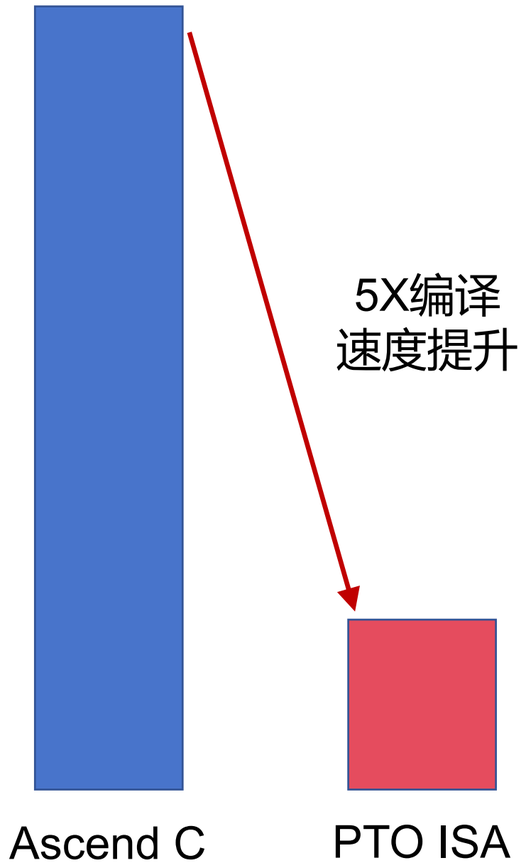
- 通过CA Model, MSPERF在PTO ISA上性能调优
- 提供C-V分离，STARS, AI CPU，CCU等底层硬件调用接口
- 提供在昇腾950的性能优化指导，帮助客户构建高性能底层

Agent时代 PTO ISA的优势

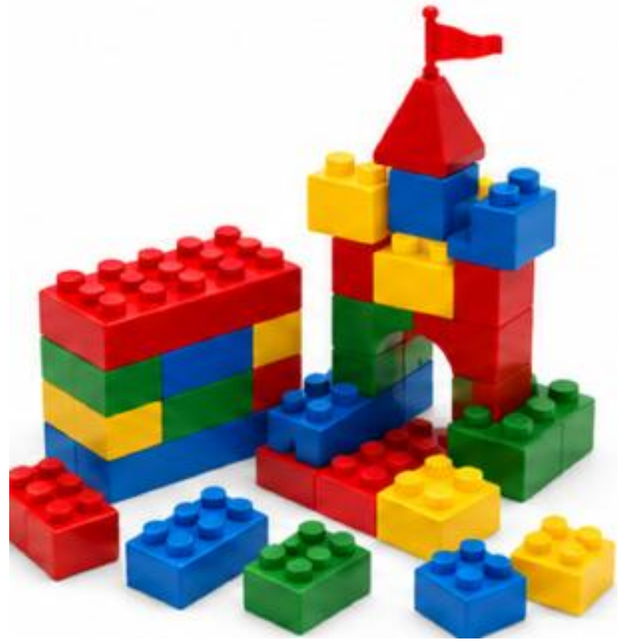
对昇腾硬件的形式化描述 掌控硬件细节



编译时间大幅降低 使能高效Tuning



收缩寻优空间 有限制调优



PyPTO算子编程专场

CANN开发者Meetup

2026.10.24　中国·上海

主办单位： CANN开源社区　**协办单位：** AtomGit

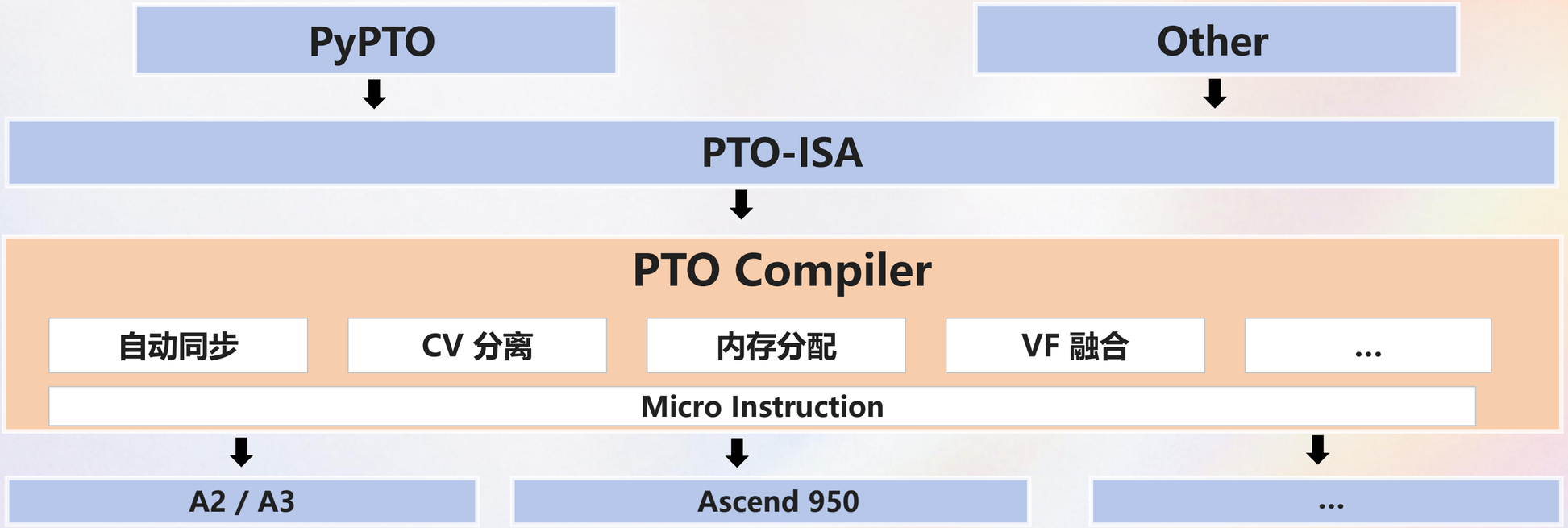


PTO编译： 面向Tile抽象的编译优化技术

自得



PTO ISA技术栈架构



自动同步

数据依赖分析

CV 分离

数据交接与同步

内存分配

规划片上存储

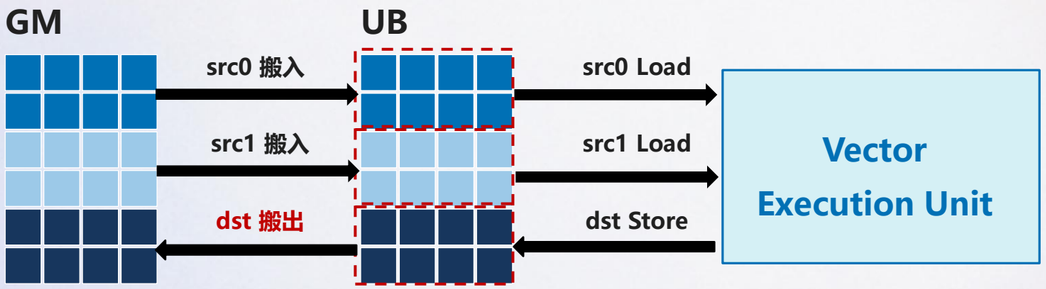
VF 融合

向量计算与数据流融合

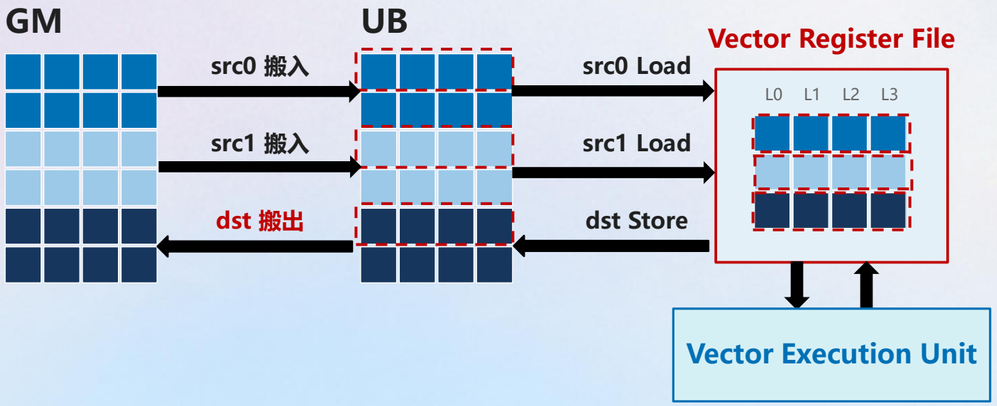
CANN

编译器视角下的 Tile 编程

A2 / A3 （Memory-based）



Ascend 950 （Register-based）



PTO-ISA实现（伪代码）

```
TAdd(dst, src0, src1):
    VF {
        for i in range(0, rows * cols, VL):
            v0 = vlds(src0 + i)
            v1 = vlds(src1 + i)
            v2 = vadd(v0, v1)
            vsts(v2, dst + i)
    }
```

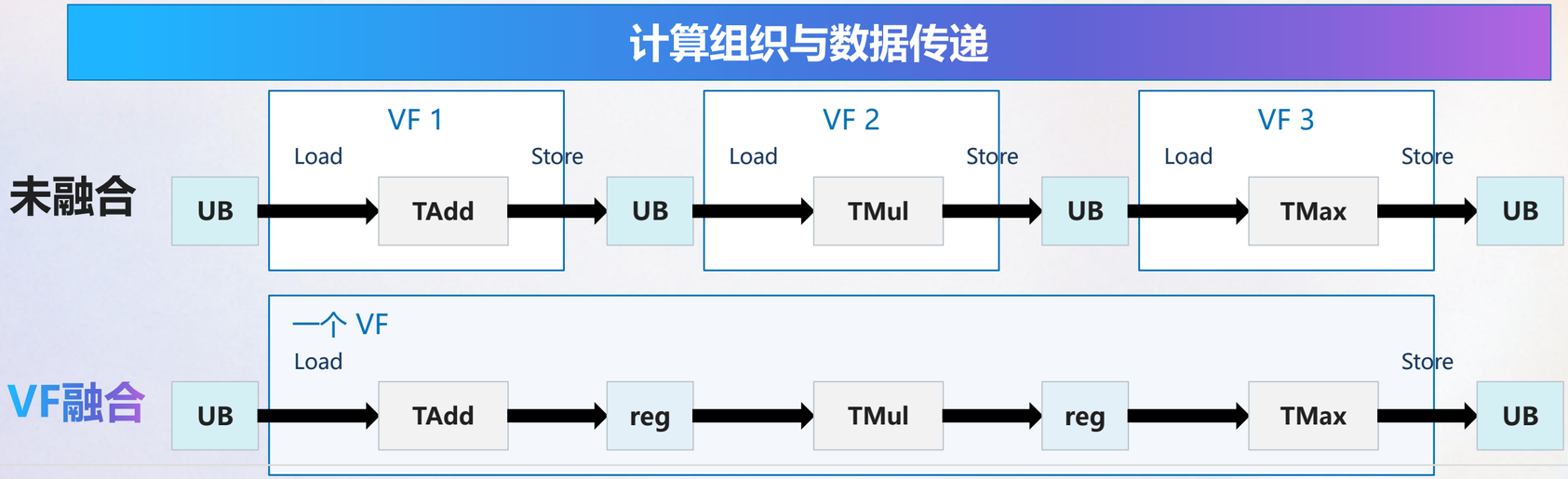
那么，一条计算链上的多个 OP 呢?

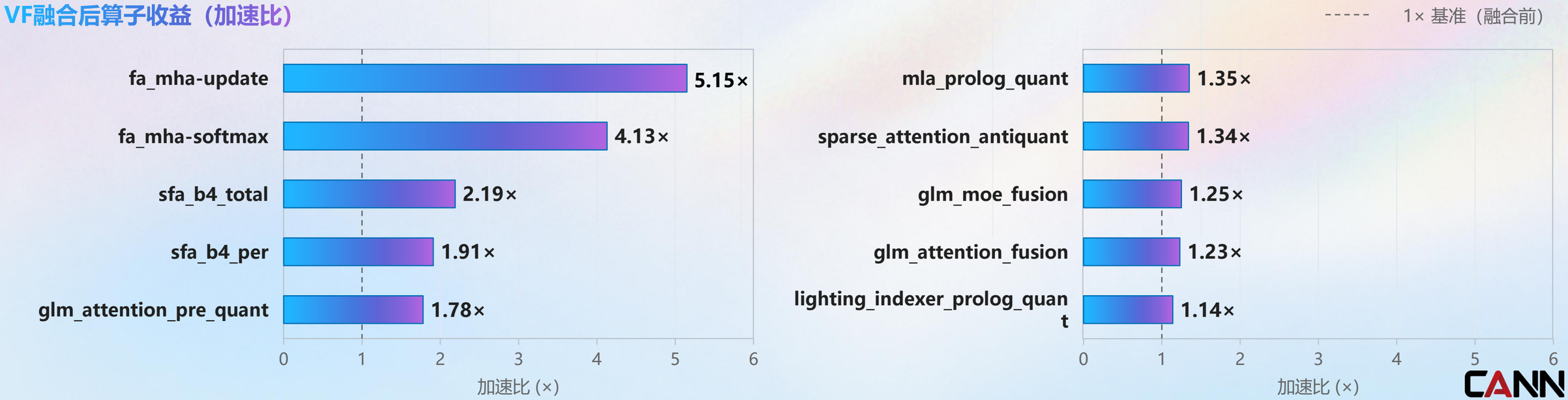
TAdd	vlds(X)	→	vadd	→	vsts(T1)
TMul	vlds(T1)	→	vmul	→	vsts(T2)
TMax	vlds(T2)	→	vmax	→	vsts(Y)

VF 融合：让中间值留在寄存器里，不再往返 UB

CANN

VF融合


计算组织与数据传递
启动与中间数据
未融合
VF 1
Load
Store
VF 2
Load
Store
VF 3
Load
Store
UB
TAdd
UB
TMul
UB
TMax
UB
3 次启动
中间值经 UB
VF融合
一个 VF
Load
Store
UB
TAdd
reg
TMul
reg
TMax
UB
1 次启动
中间值走寄存器


VF融合后算子收益（加速比）
1× 基准（融合前）
fa_mha-update
5.15×
fa_mha-softmax
4.13×
sfa_b4_total
2.19×
sfa_b4_per
1.91×
glm_attention_pre_quant
1.78×
mla_prolog_quant
1.35×
sparse_attention_antiquant
1.34×
glm_moe_fusion
1.25×
glm_attention_fusion
1.23×
lighting_indexer_prolog_quant
1.14×
0
1
2
3
4
5
6
加速比 (×)

VF融合遇到的问题

Q1

怎么正确融合?

判断两个 OP 的 UB 访问是否存在空间重叠

↓

Alias Analyze

Static | **Dynamic**

MayAlias 时必须保守拒绝，否则有正确性问题

Q2

如何寻找融合机会?

把“挡路”的 指令上提或下沉
把能融合的OP调度到一起

↓

Hoist / Sink

Schedule

调整 OP 顺序，让同 Shape 的计算连续排列

Q3

如何控制融合决策?

融合收益，需要模型预测再决定

↓

UnrollCostModel

Version Select

按依赖、延迟与寄存器压力，选展开与调度方案

CANN

Version Select

加入 fusion_group ?

	否 → 单 OP 性能最优版本	是 → 融合友好版本

循环形态
1D / 2D
	1D • 单层循环
	展平行列，单 OP 更快
	VF {
	for off in range(0, rows*cols, VL):
	a = Load(A + off)
	b = Load(B + off)
	Store(Y + off, Add(a, b))
	}
		2D • 行列两层循环
		保留行列边界，易与 Reduce / Expand 对齐
		VF {
		for row in range(rows):
		for col in range(0, cols, VL):
		a = Load(A[row] + col)
		b = Load(B[row] + col)
		Store(Y[row] + col, Add(a, b))
		}

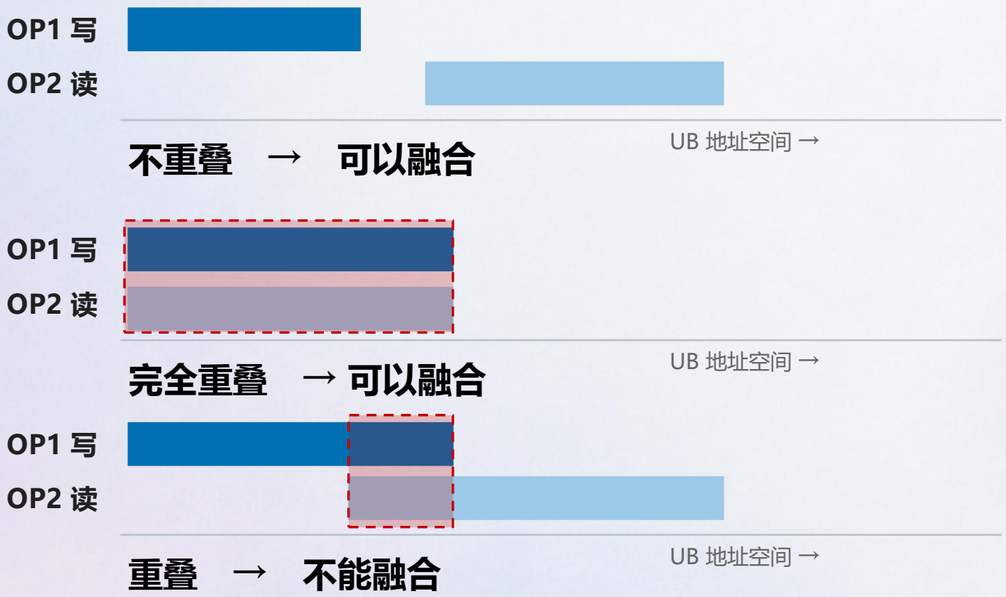
归约算法
以 ColSum 为例
	版本 A • 二分规约
	循环次数 64 → 32 → … → 1
	VF {
	for level in range(log2(rows)):
	n = rows >> (level + 1)
	for k in range(n):
	T[k] = Add(T[2k], T[2k+1])
	Store(S, T[0])
	}
		版本 B • 逐行累加
		循环次数 = rows，与上游 Exp 对齐
		VF {
		acc = 0
		for row in range(rows):
		acc = Add(acc, Load(E + row))
		Store(S, acc)
		}

CANN

Alias Analyze

融合会改变 UB 的生命周期，前提是两个 OP 的访问互不干扰

要回答的问题：两个 OP 的 UB 访问是否重叠



三种判定结果

NoAlias	确定不重叠	➡ 能融合
MustAlias	完全重叠	➡ 能融合（能消除读写）
MayAlias • PartialAlias	无法确定/部分重叠	➡ 保守拒绝

静态推导　编译期

- 按 UB 首地址 + 大小区间判断
- 地址为常量或可推导的仿射表达式时有效
- 推导不出结论，即落入 MayAlias

动态推导　结合运行期信息

- shape / stride / tile 划分确定后再判
- 把一部分 MayAlias 收敛为NoAlias
- 代价：需要额外信息或运行期检查

CANN

Hoist / Sink

Sink　下沉不影响消费者的 Store

```
OP1 {
  for row in rows {
    acc = Add(acc, Load T[row])
  }
  Store S, acc      // 隔在两个循环之间
}
OP2 {
  for row in rows {
    Store OUT[row], Cast(Load T[row])
  }
}
```

⬇ Store S 写的是 S，OP2 读的是 T

```
OP1 {
  for row in rows {
    acc = Add(acc, Load T[row])
  }
}
OP2 {
  for row in rows {
    Store OUT[row], Cast(Load T[row])
  }
}
Store S, acc      // 下沉后两循环相邻
```

Hoist　统一上提重复的地址计算

```
for row in rows {
  addr = row * cols + col
  x = Load X[addr]
}
for row in rows {
  addr = row * cols + col
  t = Load T[addr]
}
```

⬇ 地址与 row / col 有关，与循环体无关

```
addr = VAG(row, col, cols)   // 地址生成指令
for row in rows {
  x = Load X[addr[row]]
}
for row in rows {
  t = Load T[addr[row]]
}
```

CANN

Schedule

调度前　OP 顺序

MulS	ColMax	Expand	Sub	Exp	ColSum	Cast
[128, 64]	[1, 64]	[128, 64]	[128, 64]	[128, 64]	[1, 64]	[128, 64]

Shape 在中间反复突变，融合节奏被切断

调度后　MulS 下放，Cast 前移

ColMax	MulS(m)	MulS(X)	Expand	Sub	Exp	Cast	ColSum
[1, 64]	[1, 64]	[128, 64]	[128, 64]	[128, 64]	[128, 64]	[128, 64]	[1, 64]

可融合群

① MulS 下放到 ColMax 之后

小向量多算一次，换 128 行整块 MulS 进入融合群

② Cast 与 ColSum 对调

Shape 突变放到两端，同 Shape 的计算连续排列

CANN

LoopFusion / LoadStoreElimination

合并 VF 后：两个循环

```
VF {
  for i:                          TAdd
    a0 = Load a[i]
    b0 = Load b[i]
    t = VAdd(a0, b0)
    Store tmp[i], t
  for i:                          TMul
    t = Load tmp[i]
    c0 = Load c[i]
    y = VMul(t, c0)
    Store out[i], y
}
```

① 循环融合

```
VF {
  for i:                          TAdd_TMul
    a0 = Load a[i]
    b0 = Load b[i]
    t = VAdd(a0, b0)
    Store tmp[i], t
    t = Load tmp[i]
    c0 = Load c[i]
    y = VMul(t, c0)
    Store out[i], y
}
```

② Load / Store 消除

```
VF {
  for i:                          TAdd_TMul
    a0 = Load a[i]
    b0 = Load b[i]
    t = VAdd(a0, b0)
    c0 = Load c[i]
    y = VMul(t, c0)
    Store out[i], y
}
```

合并 VF 减少启动次数；循环融合创造局部性；LoadStore消除将中间传递提升为寄存器复用

CANN

Unroll + CostModel

候选展开与指令重排的 latency 预测

展开次数 × 指令顺序	
原循环	$A \rightarrow B$
×2 原序	$A_0\ B_0\ \ A_1\ B_1$
×2 重排	$A_0\ A_1\ \ B_0\ B_1$
×4 原序	$A_0\ B_0\ \ A_1\ B_1\ \ A_2\ B_2\ \ A_3\ B_3$
×4 重排	$A_0\ A_1\ A_2\ A_3\ \ B_0\ B_1\ B_2\ B_3$

…

VfSim 如何选

生成 unroll / schedule 候选

⬇

按依赖图模拟 issue 与 latency

⬇

估算 VF completion cycle

⬇

CostModel 选择低 latency 方案

展开越大不一定越快：既看指令能否并行，也看寄存器压力与依赖等待

CANN

总结

硬件给出可能性，编译器把它兑现

01

融合的空间是硬件给的

Ascend 950的编程模型为VF融合特性提供了优化前提

02

正确性、融合机会、收益权衡，三者缺一不可

判不准就不敢融合，机会没腾出来就无从融合，算不清收益就不该融合。

03

编译器自动识别优化

算子照常按 PTO-ISA Tile编程写，可融合的区段由编译器识别并完成，
已在 FA、mHC、RMSNorm 等典型算子上验证。

CANN

CANN

THANKS



CANN